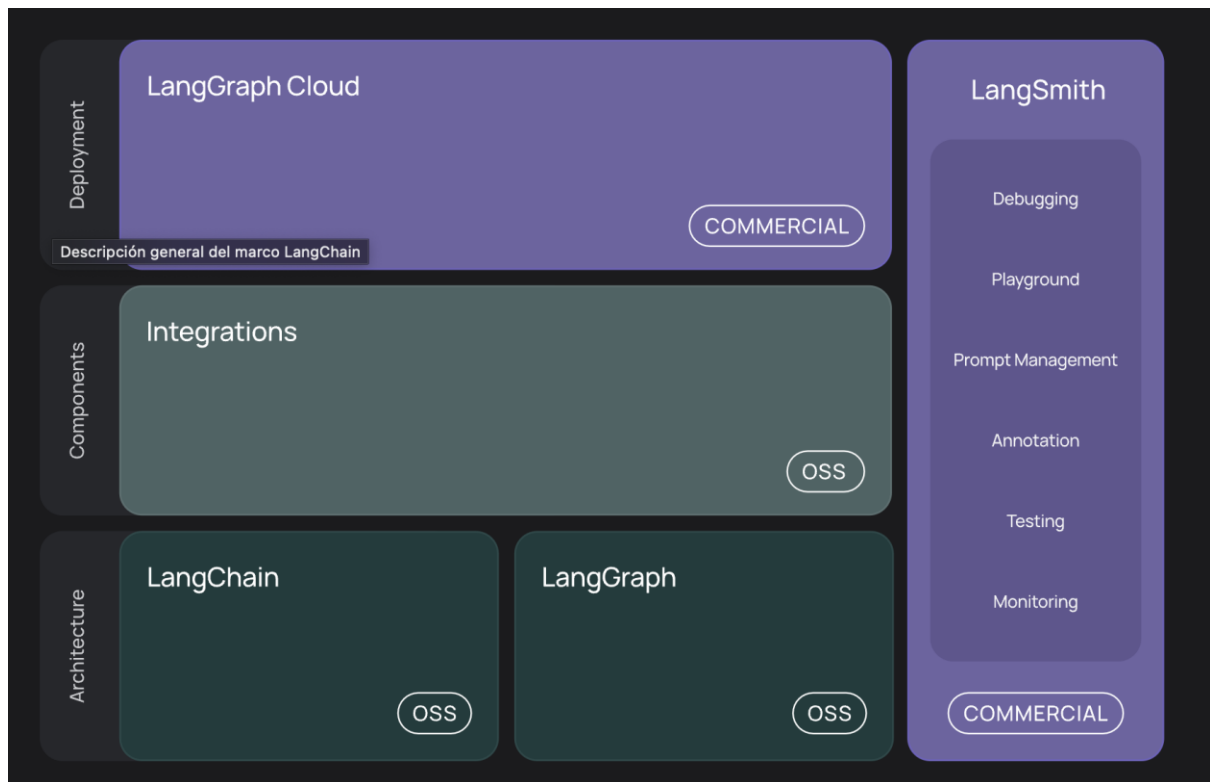


LangChain

Introducción



Concretamente, el framework consta de las siguientes librerías de código abierto:

langchain-core: Abstracciones base y lenguaje de expresión LangChain.

langchain-community: Integraciones de terceros.

Paquetes de socios (por ejemplo langchain-openai, langchain-anthropic, , etc.): Algunas integraciones se han dividido en sus propios paquetes livianos que solo dependen de langchain-core.

langchain: Cadenas, agentes y estrategias de recuperación que conforman la arquitectura cognitiva de una aplicación.

LangGraph : cree aplicaciones multiactor sólidas y con estado con LLM modelando pasos como bordes y nodos en un gráfico. Se integra sin problemas con LangChain, pero se puede utilizar sin él.

LangServe : Implementa cadenas LangChain como API REST.

LangSmith : una plataforma para desarrolladores que le permite depurar, probar, evaluar y monitorear aplicaciones LLM.

Arquitectura

LangChain como marco consta de varios paquetes.

langchain-core

Este paquete contiene abstracciones básicas de diferentes componentes y formas de combinarlos. Aquí se definen las interfaces para los componentes principales, como LLM, almacenes de vectores, recuperadores y más. No se definen integraciones de terceros. Las dependencias se mantienen muy livianas a propósito.

Paquetes para socios

Si bien la cola larga de integraciones está en langchain-community, dividimos las integraciones populares en sus propios paquetes (por ejemplo langchain-openai, langchain-anthropic, , etc.). Esto se hizo para mejorar el soporte para estas integraciones importantes.

Langchain

El paquete principal langchain contiene cadenas, agentes y estrategias de recuperación que conforman la arquitectura cognitiva de una aplicación. NO son integraciones de terceros. Todas las cadenas, agentes y estrategias de recuperación que se incluyen aquí NO son específicos de ninguna integración, sino que son genéricos para todas las integraciones.

langchain-community

Este paquete contiene integraciones de terceros que son mantenidas por la comunidad LangChain. Los paquetes de socios clave están separados (ver a continuación). Contiene todas las integraciones para varios componentes (LLM, almacenes de vectores, recuperadores). Todas las dependencias de este paquete son opcionales para mantener el paquete lo más liviano posible.

Langgraph

langgraph es una extensión langchain destinada a construir aplicaciones multiactor robustas y con estado con LLM modelando pasos como aristas y nodos en un gráfico.

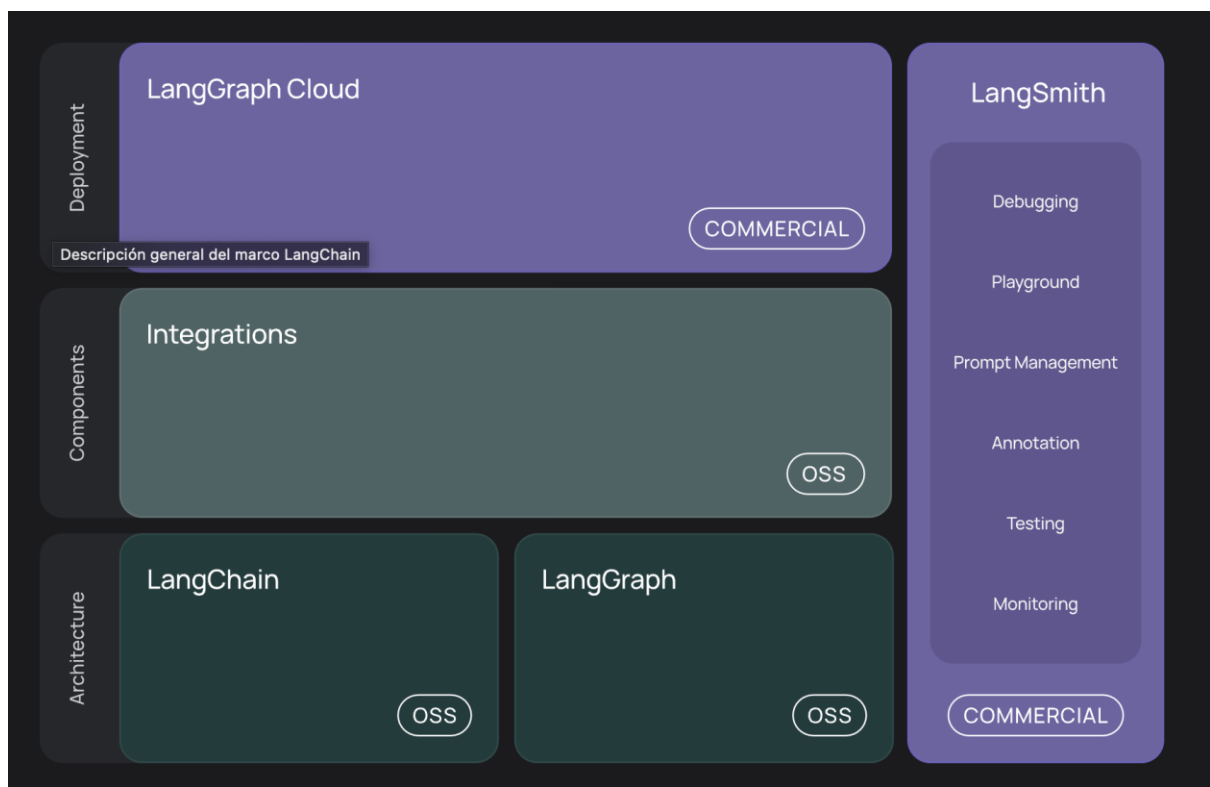
LangGraph expone interfaces de alto nivel para crear tipos comunes de agentes, así como una API de bajo nivel para componer flujos personalizados.

Langserve

Un paquete para implementar cadenas LangChain como API REST. Facilita la puesta en funcionamiento de una API lista para producción.

Langsmith

Una plataforma para desarrolladores que le permite depurar, probar, evaluar y supervisar aplicaciones LLM.



Lenguaje de expresión LangChain (LCEL)

El lenguaje de expresión LangChain, o LCEL, es una forma declarativa de encadenar componentes LangChain. LCEL se diseñó desde el primer día para permitir la puesta en producción de prototipos, sin cambios en el código, desde la cadena más simple de “solicitud + LLM” hasta las cadenas más complejas (hemos visto a gente ejecutar con éxito cadenas LCEL con cientos de pasos en producción). A continuación, se muestran algunas de las razones por las que podría querer usar LCEL:

Soporte de transmisión de primera clase Cuando creas tus cadenas con LCEL, obtienes el mejor tiempo posible hasta el primer token (tiempo transcurrido hasta que sale el primer fragmento de salida). Para algunas cadenas, esto significa, por ejemplo, que transmitimos tokens directamente desde un LLM a un analizador de salida de transmisión y que obtienes fragmentos de salida incrementales y analizados a la misma velocidad a la que el proveedor de LLM genera los tokens sin procesar.

Compatibilidad asincrónica Cualquier cadena creada con LCEL puede ser invocada tanto con la API sincrónica (por ejemplo, en su notebook Jupyter mientras crea prototipos) como con la API asincrónica (por ejemplo, en un servidor LangServe). Esto permite usar el mismo código para prototipos y en producción, con un gran rendimiento y la capacidad de manejar muchas solicitudes concurrentes en el mismo servidor.

Ejecución paralela optimizada Siempre que sus cadenas LCEL tengan pasos que se puedan ejecutar en paralelo (por ejemplo, si obtiene documentos de varios recuperadores), lo hacemos automáticamente, tanto en las interfaces sincrónicas como asincrónicas, para lograr la menor latencia posible.

Reintentos y alternativas Configure reintentos y alternativas para cualquier parte de su cadena LCEL. Esta es una excelente manera de hacer que sus cadenas sean más confiables a escala. Actualmente, estamos trabajando para agregar compatibilidad con transmisión para reintentos y alternativas, de modo que pueda obtener la confiabilidad adicional sin ningún costo de latencia.

Acceder a los resultados intermedios En el caso de cadenas más complejas, suele ser muy útil acceder a los resultados de los pasos intermedios incluso antes de que se produzca el resultado final. Esto se puede utilizar para que los usuarios finales sepan que algo está sucediendo o incluso simplemente para depurar la cadena. Puede transmitir resultados intermedios y está disponible en todos los servidores LangServe.

Esquemas de entrada y salida Los esquemas de entrada y salida proporcionan a cada cadena LCEL esquemas Pydantic y JSONSchema inferidos a partir de la

estructura de su cadena. Esto se puede utilizar para la validación de entradas y salidas, y es una parte integral de LangServe.

Seguimiento continuo de LangSmith A medida que las cadenas se vuelven cada vez más complejas, se vuelve cada vez más importante comprender qué sucede exactamente en cada paso. Con LCEL, todos los pasos se registran automáticamente en LangSmith para lograr la máxima capacidad de observación y depuración.

LCEL tiene como objetivo brindar coherencia en el comportamiento y la personalización de cadenas de subclases heredadas, como LLMChainy ConversationalRetrievalChain. Muchas de estas cadenas heredadas ocultan detalles importantes, como indicaciones, y a medida que surge una mayor variedad de modelos viables, la personalización se ha vuelto cada vez más importante.

Si actualmente está utilizando una de estas cadenas heredadas, consulte esta guía para obtener orientación sobre cómo migrar .

Para obtener guías sobre cómo realizar tareas específicas con LCEL, consulte las guías prácticas correspondientes .

Interfaz Runnable

Para que sea lo más fácil posible crear cadenas personalizadas, hemos implementado un protocolo "Runnable" . Muchos componentes de LangChain implementan el Runnable protocolo, incluidos los modelos de chat, los LLM, los analizadores de salida, los recuperadores, las plantillas de mensajes y más. También hay varios primitivos útiles para trabajar con ejecutables, sobre los que puede leer a continuación.

Se trata de una interfaz estándar que facilita la definición de cadenas personalizadas y su invocación de forma estándar. La interfaz estándar incluye:

- **Stream:** transmitir fragmentos de la respuesta
- **Invoke:** llamar a la cadena en una entrada
- **Batch:** llamar a la cadena en una lista de entradas

Estos también tienen métodos asincrónicos correspondientes que deben usarse con la sintaxis asyncio await para concurrencia:

- **Astream:** transmitir fragmentos de la respuesta de forma asincrónica
- **Ainvoke:** llamar a la cadena en una entrada asíncrona

- **Abatch:** llamar a la cadena en una lista de entradas asíncronas
- **Astream_log:** retransmitir los pasos intermedios a medida que ocurren, además de la respuesta final
- **Astream_events:** eventos de transmisión beta a medida que ocurren en la cadena (introducido en langchain-core0.1.14)

El tipo de entrada y el tipo de salida varían según el componente:

Component	Input Type	Output Type
Prompt	Dictionary	PromptValue
ChatModel	Single string, list of chat messages or a PromptValue	ChatMessage
LLM	Single string, list of chat messages or a PromptValue	String
OutputParser	The output of an LLM or ChatModel	Depends on the parser
Retriever	Single string	List of Documents
Tool	Single string or dictionary, depending on the tool	Depends on the tool

Todos los ejecutables exponen esquemas de entrada y salida para inspeccionar las entradas y salidas:

- **input_schema:** un modelo Pydantic de entrada generado automáticamente a partir de la estructura del Runnable
- **output_schema:** un modelo de Pydantic de salida generado automáticamente a partir de la estructura del Runnable

Componentes

LangChain ofrece interfaces estándar y extensibles e integraciones externas para varios componentes útiles para la creación con LLM. Algunos componentes los implementa LangChain, otros los integramos con integraciones de terceros y otros son una combinación.

Modelos de chat

Modelos de lenguaje que utilizan una secuencia de mensajes como entradas y devuelven mensajes de chat como salidas (en lugar de utilizar texto simple). Estos son modelos tradicionalmente más nuevos (los modelos más antiguos son generalmente LLMs, consulte a continuación). Los modelos de chat admiten la asignación de roles distintos a los mensajes de conversación, lo que ayuda a distinguir los mensajes de la IA, los usuarios y las instrucciones, como los mensajes del sistema.

Aunque los modelos subyacentes son mensajes de entrada y mensajes de salida, los envoltorios LangChain también permiten que estos modelos tomen una cadena como entrada. Esto significa que puede usar fácilmente modelos de chat en lugar de LLM.

Cuando se pasa una cadena como entrada, se convierte en un `HumanMessage` y luego se pasa al modelo subyacente.

Tenemos algunos parámetros estandarizados al construir `ChatModels`:

- **model:** el nombre del modelo
- **temperature:** la temperatura de muestreo
- **timeout:** tiempo de espera de solicitud
- **max_tokens:** máximo de tokens a generar
- **stop:** secuencias de parada predeterminadas
- **max_retries:** número máximo de veces que se pueden reintentar las solicitudes
- **api_key:** Clave API para el proveedor de modelos
- **base_url:** punto final al que enviar solicitudes

Algunas cosas importantes a tener en cuenta:

- Los parámetros estándar solo se aplican a los proveedores de modelos que exponen parámetros con la funcionalidad prevista. Por ejemplo, algunos proveedores no exponen una configuración para la cantidad máxima de tokens de salida, por lo que no se puede admitir `max_tokens` en ellos.
- Actualmente, los parámetros estándar solo se aplican en integraciones que tienen sus propios paquetes de integración (por ejemplo `langchain-openai`, `langchain-anthropic`, , etc.), no se aplican en modelos en `langchain-community`.

Los ChatModels también aceptan otros parámetros específicos de esa integración. Para encontrar todos los parámetros admitidos por un ChatModel, diríjase a la referencia de API de ese modelo.

información

Algunos modelos de chat se han perfeccionado para la llamada de herramientas y proporcionan una API dedicada para ello. Por lo general, estos modelos son mejores para la llamada de herramientas que los modelos que no están perfeccionados y se recomiendan para casos de uso que requieren la llamada de herramientas. Consulta la sección de llamada de herramientas para obtener más información.

Para obtener información específica sobre cómo utilizar los modelos de chat, consulte las guías prácticas correspondientes aquí.

Multimodalidad.

Algunos modelos de chat son multimodales y aceptan imágenes, audio e incluso video como entradas. Estos son aún menos comunes, lo que significa que los proveedores de modelos no han estandarizado la "mejor" forma de definir la API. Las salidas multimodales son aún menos comunes. Por eso, hemos mantenido nuestras abstracciones multimodales bastante livianas y planeamos consolidar aún más las API multimodales y los patrones de interacción a medida que el campo madure.

En LangChain, la mayoría de los modelos de chat que admiten entradas multimodales también aceptan esos valores en el formato de bloques de contenido de OpenAI. Hasta ahora, esto está restringido a las entradas de imágenes. En el caso de modelos como Gemini, que admiten entradas de video y otros bytes, las API también admiten las representaciones nativas específicas del modelo.

Para obtener información específica sobre cómo utilizar modelos multimodales, consulte las guías prácticas pertinentes aquí .

Para obtener una lista completa de proveedores de modelos LangChain con modelos multimodales, consulte esta tabla.

LLMs

precaución.

Los modelos LLM de entrada y salida de texto puros tienden a ser más antiguos o de nivel inferior. Muchos modelos populares se utilizan mejor como modelos de finalización de chat , incluso para casos de uso que no sean de chat.

Probablemente estás buscando la sección de arriba .

Modelos de lenguaje que toman una cadena como entrada y devuelven una cadena. Estos son modelos tradicionalmente más antiguos (los modelos más nuevos generalmente son modelos de chat , consulte más arriba).

Aunque los modelos subyacentes son string in, string out, los envoltorios LangChain también permiten que estos modelos tomen mensajes como entrada. Esto les proporciona la misma interfaz que los modelos de chat . Cuando se pasan mensajes como entrada, se les dará formato en una cadena en segundo plano antes de pasarlos al modelo subyacente.

LangChain no aloja ningún LLM, sino que confiamos en integraciones de terceros.

Para obtener información específica sobre cómo utilizar los LLM, consulte las guías prácticas pertinentes aquí.

Message

Algunos modelos de lenguaje toman una lista de mensajes como entrada y devuelven un mensaje. Existen distintos tipos de mensajes. Todos los mensajes tienen una propiedad role, content y response_metadata.

Describe role QUIÉN transmite el mensaje. LangChain tiene diferentes clases de mensajes para diferentes roles.

La content propiedad describe el contenido del mensaje. Puede ser de varias formas:

- Una cadena (la mayoría de los modelos manejan este tipo de contenido)
- Una lista de diccionarios (se utiliza para entrada multimodal, donde el diccionario contiene información sobre ese tipo de entrada y esa ubicación de entrada)

HumanMessage

Esto representa un mensaje del usuario.

AIMessage

Esto representa un mensaje del modelo. Además de la content propiedad, estos mensajes también tienen:

response_metadata

La response_metadata propiedad contiene metadatos adicionales sobre la respuesta. Los datos que se encuentran aquí suelen ser específicos de cada proveedor de modelos. Aquí es donde se puede almacenar información como log-probs y uso de tokens.

tool_calls

Representan una decisión de un modelo de lenguaje para llamar a una herramienta. Se incluyen como parte de una AIMessage salida. Se puede acceder a ellos desde allí con la .tool_calls propiedad.

Esta propiedad devuelve una lista de ToolCalls. Un ToolCall es un diccionario con los siguientes argumentos:

- **name:** El nombre de la herramienta que debe llamarse.
- **args:** Los argumentos de esa herramienta.
- **id:** El identificador de esa llamada de herramienta.

SystemMessage

Esto representa un mensaje del sistema que le indica al modelo cómo debe comportarse. No todos los proveedores de modelos lo admiten.

ToolMessage

Esto representa el resultado de una llamada a una herramienta. Además de role y content, este mensaje tiene:

- un tool_call_id campo que transmite el identificador de la llamada a la herramienta que se llamó para producir este resultado.
- un artifact campo que puede usarse para pasar artefactos arbitrarios de la ejecución de la herramienta que son útiles para rastrear pero que no deben enviarse al modelo.

(Legacy) FunctionMessage

Este es un tipo de mensaje heredado, que corresponde a la API de llamada de funciones heredadas de OpenAI. ToolMessage debe usarse en su lugar para corresponder a la API de llamada de herramientas actualizada.

Representa el resultado de una llamada a una función. Además de role y content, este mensaje tiene un name parámetro que transmite el nombre de la función que se llamó para producir este resultado.

Prompt templates

Las plantillas de indicaciones ayudan a traducir la entrada y los parámetros del usuario en instrucciones para un modelo de lenguaje. Esto se puede utilizar para guiar la respuesta de un modelo, ayudándolo a comprender el contexto y a generar una salida relevante y coherente basada en el lenguaje.

Las plantillas de solicitud toman como entrada un diccionario, donde cada clave representa una variable en la plantilla de solicitud para completar.

Las plantillas de mensajes generan un PromptValue. Este PromptValue se puede pasar a un LLM o a un ChatModel, y también se puede convertir en una cadena o en una lista de mensajes. La razón por la que existe este PromptValue es para facilitar el cambio entre cadenas y mensajes.

Hay algunos tipos diferentes de plantillas de indicaciones:

String PromptTemplates

Estas plantillas de mensajes se utilizan para dar formato a una sola cadena y, por lo general, se utilizan para entradas más simples. Por ejemplo, una forma común de construir y utilizar una plantilla de mensajes es la siguiente:

```
from langchain_core.prompts import PromptTemplate
```

```
prompt_template = PromptTemplate.from_template("Tell me a joke  
about {topic}")
```

```
prompt_template.invoke({"topic": "cats"})
```

ChatPromptTemplates

Estas plantillas de mensajes se utilizan para dar formato a una lista de mensajes. Estas "plantillas" consisten en una lista de plantillas. Por ejemplo, una forma habitual de construir y utilizar una ChatPromptTemplate es la siguiente:

```
from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    ("user", "Tell me a joke about {topic}")
])

prompt_template.invoke({"topic": "cats"})
```

En el ejemplo anterior, esta ChatPromptTemplate construirá dos mensajes cuando se la llame. El primero es un mensaje del sistema, que no tiene variables para formatear. El segundo es un HumanMessage, y se formateará según la topic variable que el usuario ingrese.

MessagesPlaceholder

Esta plantilla de mensaje es responsable de agregar una lista de mensajes en un lugar determinado. En la plantilla ChatPromptTemplate anterior, vimos cómo podríamos dar formato a dos mensajes, cada uno de ellos una cadena. Pero, ¿qué sucede si queremos que el usuario pase una lista de mensajes que colocaremos en un lugar determinado? Así es como se usa MessagesPlaceholder.

```
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.messages import HumanMessage

prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    MessagesPlaceholder("msgs")
])

prompt_template.invoke({"msgs": [HumanMessage(content="hi!")]})
```

Esto producirá una lista de dos mensajes, el primero será un mensaje del sistema y el segundo será el HumanMessage que pasamos. Si hubiéramos pasado 5 mensajes, entonces habría producido 6 mensajes en total (el mensaje del sistema

más los 5 pasados). Esto es útil para permitir que una lista de mensajes se coloque en un lugar en particular.

Una forma alternativa de lograr lo mismo sin usar la `MessagesPlaceholder` clase explícitamente es:

```
prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    ("placeholder", "{msgs}") # <-- This is the changed part
])
```

Para obtener información específica sobre cómo utilizar las plantillas de indicaciones, consulte las guías prácticas pertinentes [aquí](#) .

Example selectors

Una técnica de solicitud común para lograr un mejor rendimiento es incluir ejemplos como parte de la solicitud. Esto le da al modelo de lenguaje ejemplos concretos de cómo debería comportarse. A veces, estos ejemplos están codificados en la solicitud, pero para situaciones más avanzadas puede ser bueno seleccionarlos dinámicamente. Los selectores de ejemplos son clases responsables de seleccionar y luego formatear ejemplos en las solicitudes.

Para obtener información específica sobre cómo utilizar selectores de ejemplo, consulte las guías prácticas correspondientes [aquí](#) .

Output parsers

NOTA: La información aquí se refiere a los analizadores que toman una salida de texto de un modelo e intentan analizarla para obtener una representación más estructurada. Cada vez más modelos admiten llamadas a funciones (o herramientas), que se encargan de esto automáticamente. Se recomienda utilizar llamadas a funciones o herramientas en lugar de análisis de salida. Consulte la documentación al respecto [aquí](#) .

Historial de chat

La mayoría de las aplicaciones de LLM tienen una interfaz conversacional. Un componente esencial de una conversación es poder hacer referencia a la información presentada anteriormente en la conversación. Como mínimo, un sistema conversacional debería poder acceder directamente a alguna ventana de mensajes anteriores.

El concepto de ChatHistory se refiere a una clase en LangChain que se puede utilizar para encapsular una cadena arbitraria. Esto ChatHistory hará un seguimiento de las entradas y salidas de la cadena subyacente y las agregará como mensajes a una base de datos de mensajes. Las interacciones futuras cargarán esos mensajes y los pasarán a la cadena como parte de la entrada.

Documents

Un objeto de documento en LangChain contiene información sobre algunos datos. Tiene dos atributos:

- `page_content`: str: El contenido de este documento actualmente es solo una cadena.
- `metadata`: dict: Metadatos arbitrarios asociados con este documento. Pueden rastrear el ID del documento, el nombre del archivo, etc.

Document loaders

Estas clases cargan objetos Document. LangChain tiene cientos de integraciones con varias fuentes de datos para cargar datos desde: Slack, Notion, Google Drive, etc.

Cada DocumentLoader tiene sus propios parámetros específicos, pero todos pueden invocarse de la misma manera con el `.load` método. A continuación se muestra un ejemplo de caso de uso:

```
from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader(
    ... # <-- Integration specific parameters here
)
```

```
data = loader.load()
```

Para obtener información específica sobre cómo utilizar los cargadores de documentos, consulte las guías prácticas correspondientes [aquí](#) .

Text splitters

Una vez que haya cargado los documentos, con frecuencia querrá transformarlos para que se adapten mejor a su aplicación. El ejemplo más simple es que puede querer dividir un documento largo en fragmentos más pequeños que puedan caber en la ventana de contexto de su modelo. LangChain tiene una serie de transformadores de documentos integrados que facilitan la división, combinación, filtrado y manipulación de documentos.

Cuando se trata de textos extensos, es necesario dividir el texto en fragmentos. Aunque parezca sencillo, esto puede entrañar una gran complejidad. Lo ideal es mantener juntos los fragmentos de texto relacionados semánticamente. El significado de "relacionado semánticamente" puede depender del tipo de texto. Este cuaderno muestra varias formas de hacerlo.

En un nivel alto, los divisores de texto funcionan de la siguiente manera:

1. Divida el texto en fragmentos pequeños y semánticamente significativos (a menudo oraciones).
2. Comience a combinar estos pequeños trozos en un trozo más grande hasta alcanzar un tamaño determinado (medido por alguna función).
3. Una vez que alcances ese tamaño, haz que ese fragmento sea su propio fragmento de texto y luego comienza a crear un nuevo fragmento de texto con cierta superposición (para mantener el contexto entre los fragmentos).

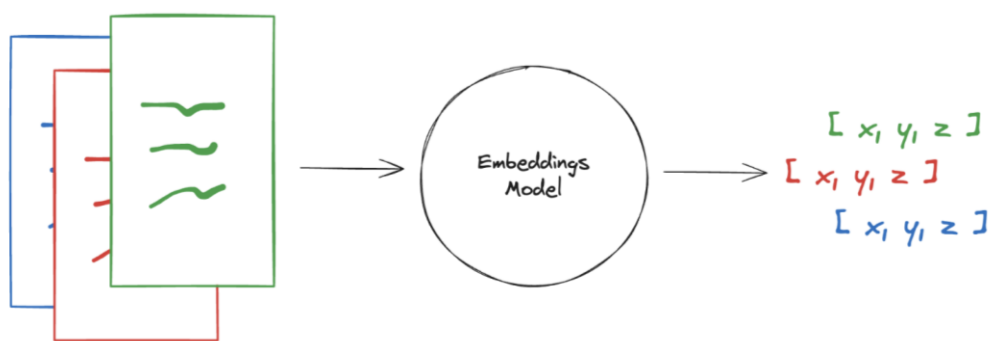
Esto significa que hay dos ejes diferentes a lo largo de los cuales puedes personalizar tu divisor de texto:

1. Cómo se divide el texto
2. Cómo se mide el tamaño del fragmento

Para obtener información específica sobre cómo utilizar divisores de texto, consulte las guías prácticas correspondientes [aquí](#).

Embedding models

Los modelos de incrustación crean una representación vectorial de un fragmento de texto. Puede pensar en un vector como una matriz de números que captura el significado semántico del texto. Al representar el texto de esta manera, puede realizar operaciones matemáticas que le permiten hacer cosas como buscar otros fragmentos de texto que tengan un significado más similar. Estas capacidades de búsqueda en lenguaje natural respaldan muchos tipos de recuperación de contexto , donde proporcionamos a un LLM los datos relevantes que necesita para responder de manera eficaz a una consulta.



La `Embeddings` clase está diseñada para interactuar con modelos de incrustación de texto. Hay muchos proveedores de modelos de incrustación diferentes (OpenAI, Cohere, Hugging Face, etc.) y modelos locales, y esta clase está diseñada para proporcionar una interfaz estándar para todos ellos.

La clase base `Embeddings` de LangChain proporciona dos métodos: uno para incrustar documentos y otro para incrustar una consulta. El primero toma como entrada varios textos, mientras que el segundo toma un solo texto. La razón por la que estos son dos métodos separados es que algunos proveedores de incrustaciones tienen diferentes métodos de incrustación para documentos (en los que se realizará la búsqueda) y consultas (la consulta de búsqueda en sí).

Para obtener información específica sobre cómo utilizar modelos de inserción, consulte las guías prácticas pertinentes [aquí](#) .

Store vector

Una de las formas más comunes de almacenar y buscar datos no estructurados es incrustarlos y almacenar los vectores de incrustación resultantes y, luego, en el momento de la consulta, incrustar la consulta no estructurada y recuperar los vectores de incrustación que sean "más similares" a la consulta incrustada. Un almacén de vectores se encarga de almacenar los datos incrustados y realizar la búsqueda de vectores por usted.

La mayoría de las tiendas de vectores también pueden almacenar metadatos sobre vectores integrados y admitir el filtrado de esos metadatos antes de la búsqueda de similitud, lo que le permite un mayor control sobre los documentos devueltos.

Las tiendas vectoriales se pueden convertir a la interfaz del recuperador haciendo lo siguiente:

```
vectorstore = MyVectorStore()  
retriever = vectorstore.as_retriever()
```

Para obtener información específica sobre cómo utilizar las tiendas vectoriales, consulte las guías prácticas correspondientes [aquí](#) .

Retrievers

Un recuperador es una interfaz que devuelve documentos dada una consulta no estructurada. Es más general que un almacén de vectores. Un recuperador no necesita poder almacenar documentos, solo devolverlos (o recuperarlos). Los recuperadores se pueden crear a partir de almacenes de vectores, pero también son lo suficientemente amplios como para incluir la búsqueda de Wikipedia y Amazon Kendra .

Los recuperadores aceptan una consulta de cadena como entrada y devuelven una lista de documentos como salida.

Para obtener información específica sobre cómo utilizar retrievers, consulte las guías prácticas pertinentes [aquí](#) .

Key-value stores

Para algunas técnicas, como la indexación y recuperación con múltiples vectores por documento o el almacenamiento en caché de incrustaciones, es útil tener una forma de almacenamiento de clave-valor (KV).

LangChain incluye una `BaseStore` interfaz que permite el almacenamiento de datos arbitrarios. Sin embargo, los componentes de LangChain que requieren almacenamiento KV aceptan una `BaseStore[str, bytes]` instancia más específica que almacena datos binarios (denominada `ByteStore`) y se encargan internamente de codificar y decodificar datos para sus necesidades específicas.

Esto significa que, como usuario, usted solo necesita pensar en un tipo de tienda en lugar de en diferentes para distintos tipos de datos.

Interface

Todos `BaseStores` admiten la siguiente interfaz. Tenga en cuenta que la interfaz permite modificar varios pares clave-valor a la vez:

- **`mget(key: Sequence[str]) -> List[Optional[bytes]]`**: obtiene el contenido de múltiples claves y retorna `None` si la clave no existe
- **`mset(key_value_pairs: Sequence[Tuple[str, bytes]]) -> None`**: establecer el contenido de múltiples claves
- **`mdelete(key: Sequence[str]) -> None`**: eliminar varias claves
- **`yield_keys(prefix: Optional[str] = None) -> Iterator[str]`**: proporciona todas las claves en el almacén, filtrando opcionalmente por un prefijo

Para implementaciones de almacenamiento de valores clave, consulte esta sección .

Tools

Las herramientas son utilidades diseñadas para ser llamadas por un modelo: sus entradas están diseñadas para ser generadas por modelos y sus salidas están diseñadas para ser devueltas a los modelos. Las herramientas son necesarias siempre que se desea que un modelo controle partes del código o llame a API externas.

Una herramienta consta de:

- El nombre de la herramienta.
- Una descripción de lo que hace la herramienta.
- Un esquema JSON que define las entradas a la herramienta.
- Una función (y, opcionalmente, una variante asíncrona de la función).

Cuando una herramienta está vinculada a un modelo, el nombre, la descripción y el esquema JSON se proporcionan como contexto para el modelo. Dada una lista de herramientas y un conjunto de instrucciones, un modelo puede solicitar llamar a una o más herramientas con entradas específicas. El uso típico puede ser similar al siguiente:

```
tools = [...] # Define a list of tools
llm with tools = llm.bind tools(tools)
ai_msg = llm with tools.invoke("do xyz...")
# -> AIMessage(tool_calls=[ToolCall(...), ...], ...)
```

El AIMessage modelo devuelto PUEDE tener tool_calls asociado el tipo de respuesta. Lea esta guía para obtener más información sobre cómo puede verse el tipo de respuesta.

Una vez que se invocan las herramientas elegidas, los resultados se pueden devolver al modelo para que pueda completar la tarea que esté realizando. En general, hay dos formas diferentes de invocar la herramienta y devolver la respuesta:

Invoke with just the arguments

Cuando invocas una herramienta con solo los argumentos, obtendrás el resultado original de la herramienta (normalmente una cadena). Por lo general, se ve así:

```
# You will want to previously check that the LLM returned tool
calls
tool_call = ai_msg.tool_calls[0]
# ToolCall(args={...}, id=..., ...)
tool_output = tool.invoke(tool_call["args"])
tool_message = ToolMessage(
    content=tool_output,
    tool_call_id=tool_call["id"],
    name=tool_call["name"]
)
```

Tenga en cuenta que, por lo general, el content campo se devolverá al modelo. Si no desea que la respuesta de la herramienta sin procesar se transmita al modelo, pero desea conservarla, puede transformar la salida de la herramienta, pero también pasarla como un artefacto (obtenga más información ToolMessage.artifact aquí)

```
... # Same code as above
response for llm = transform(response)
tool_message = ToolMessage(
```

```

    content=response for llm,
    tool_call_id=tool_call["id"],
    name=tool_call["name"],
    artifact=tool_output
)

```

Invoke with ToolCall

La otra forma de invocar una herramienta es llamarla con el código completo ToolCall que generó el modelo. Al hacer esto, la herramienta devolverá un ToolMessage. Los beneficios de esto son que no tiene que escribir la lógica usted mismo para transformar la salida de la herramienta en un ToolMessage. Por lo general, esto se ve así:

```

tool_call = ai_msg.tool_calls[0]
# -> ToolCall(args={...}, id=..., ...)
tool_message = tool.invoke(tool_call)
# -> ToolMessage(
    content="tool result foobar...",
    tool_call_id=...,
    name="tool_name"
)

```

Si invoca la herramienta de esta manera y desea incluir un artefacto para ToolMessage, deberá hacer que la herramienta devuelva dos cosas. Lea más sobre cómo definir herramientas que devuelven artefactos aquí .

Best practices

Al diseñar herramientas que utilizará un modelo, es importante tener en cuenta que:

- Los modelos de chat que tienen API explícitas de llamada de herramientas serán mejores en la llamada de herramientas que los modelos no ajustados.
- Los modelos funcionarán mejor si las herramientas tienen nombres, descripciones y esquemas JSON bien elegidos. Esta es otra forma de ingeniería rápida.
- Las herramientas simples y de alcance limitado son más fáciles de usar para los modelos que las herramientas complejas.

Related

Para obtener información específica sobre cómo utilizar las herramientas, consulte las guías prácticas de herramientas .

Para utilizar una herramienta prediseñada, consulte la documentación de integración de herramientas .

Toolkits

Los kits de herramientas son conjuntos de herramientas que están diseñados para usarse en conjunto en tareas específicas. Tienen métodos de carga convenientes.

Todos los Toolkits exponen un `get_tools` método que devuelve una lista de herramientas. Por lo tanto, puedes hacer lo siguiente:

```
# Initialize a toolkit
toolkit = ExampleToolkit(...)

# Get list of tools
tools = toolkit.get_tools()
```

Agents

Por sí solos, los modelos de lenguaje no pueden realizar acciones, solo generan texto. Un gran caso de uso para LangChain es la creación de agentes . Los agentes son sistemas que utilizan un LLM como motor de razonamiento para determinar qué acciones realizar y cuáles deberían ser las entradas para esas acciones. Los resultados de esas acciones pueden luego ser devueltos al agente y este determina si se necesitan más acciones o si está bien terminar.

LangGraph es una extensión de LangChain específicamente diseñada para crear agentes altamente controlables y personalizables. Consulte la documentación para obtener una descripción más detallada de los conceptos de los agentes.

Existe un concepto de agente heredado en LangChain que estamos tratando de dejar obsoleto: `AgentExecutor` era esencialmente un entorno de ejecución para agentes. Era un excelente lugar para comenzar, sin embargo, no era lo suficientemente flexible a medida que comenzaba a tener más agentes personalizados. Para resolver eso, creamos LangGraph para que fuera este entorno de ejecución flexible y altamente controlable.

Si aún utiliza `AgentExecutor`, no se preocupe: contamos con una guía sobre cómo utilizar `AgentExecutor` . Sin embargo, se recomienda que comience a realizar la

transición a LangGraph. Para ayudarlo en este proceso, hemos elaborado una guía de transición sobre cómo hacerlo .

ReAct agents

Una arquitectura popular para crear agentes es ReAct . ReAct combina razonamiento y acción en un proceso iterativo; de hecho, el nombre "ReAct" significa "Reason" (razón) y "Act" (acción).

El flujo general se ve así:

- El modelo "pensará" sobre qué paso dar en respuesta a una entrada y a cualquier observación previa.
- Luego, el modelo elegirá una acción entre las herramientas disponibles (o elegirá responder al usuario).
- El modelo generará argumentos para esa herramienta.
- El tiempo de ejecución del agente (ejecutor) analizará la herramienta elegida y la llamará con los argumentos generados.
- El ejecutor devolverá los resultados de la llamada de la herramienta al modelo como una observación.
- Este proceso se repite hasta que el agente decide responder.
- Hay implementaciones generales basadas en indicaciones que no requieren ninguna característica específica del modelo, pero las implementaciones más confiables usan características como llamada de herramientas para formatear salidas de manera confiable y reducir la variación.

Consulte la documentación de LangGraph para obtener más información o esta guía práctica para obtener información específica sobre la migración a LangGraph.

Callbacks

LangChain ofrece un sistema de devolución de llamadas que le permite conectarse a las distintas etapas de su aplicación LLM. Esto resulta útil para el registro, la supervisión, la transmisión y otras tareas.

Puede suscribirse a estos eventos mediante el `callbacks` argumento disponible en toda la API. Este argumento es una lista de objetos de controlador, que se espera que implementen uno o más de los métodos que se describen a continuación con más detalle.

Callback Events

Event	Event Trigger	Associated Method
Chat model start	Cuando comienza un modelo de chat	on_chat_model_start
LLM start	Cuando comienza un llm	on_llm_start
LLM new token	Cuando un modelo llm OR chat emite un nuevo token	on_llm_new_token
LLM ends	Cuando termina un modelo de chat o llm	on_llm_end
LLM errors	Cuando un modelo de chat o llm tiene errores	on_llm_error
Chain start	Cuando una cadena empieza a funcionar	on_chain_start
Chain end	Cuando una cadena termina	on_chain_end
Chain error	Cuando una cadena tiene errores	on_chain_error
Tool start	Cuando una herramienta comienza a funcionar	on_tool_start
Tool end	Cuando una herramienta termina	on_tool_end
Tool error	Cuando una herramienta falla	on_tool_error
Agent action	Cuando un agente realiza una acción	on_agent_action
Agent finish	Cuando un agente termina	on_agent_finish
Retriever start	Cuando un retriever empieza	on_retriever_start
Retriever end	Cuando un retriever termina	on_retriever_end
Retriever error	Cuando un retriever comete un error	on_retriever_error
Text	Cuando se ejecuta texto arbitrario	on_text
Retry	Cuando se ejecuta un evento de reintento	on_retry

Callback handlers

Los controladores de devolución de llamada pueden ser sync o async:

- Los controladores de devolución de llamadas de sincronización implementan la interfaz BaseCallbackHandler .
- Los controladores de devolución de llamadas asíncronas implementan la interfaz AsyncCallbackHandler .

Durante el tiempo de ejecución, LangChain configura un administrador de devolución de llamadas apropiado (por ejemplo, CallbackManager o AsyncCallbackManager , que será responsable de llamar al método apropiado en cada controlador de devolución de llamadas "registrado" cuando se activa el evento.

Passing callbacks

La callbacks propiedad está disponible en la mayoría de los objetos de la API (modelos, herramientas, agentes, etc.) en dos lugares diferentes:

Las devoluciones de llamadas están disponibles en la mayoría de los objetos de la API (modelos, herramientas, agentes, etc.) en dos lugares diferentes:

Devoluciones de llamadas en el momento de la solicitud : se pasan en el momento de la solicitud además de los datos de entrada. Disponibles en todos Runnablelos objetos estándar. Estas devoluciones de llamadas son HEREDADAS por todos los elementos secundarios del objeto en el que están definidas. Por ejemplo, chain.invoke({"number": 25}, {"callbacks": [handler]}).

Devoluciones de llamadas del constructor : chain =

TheNameOfSomeChain(callbacks=[handler])estas devoluciones de llamadas se pasan como argumentos al constructor del objeto. Las devoluciones de llamadas tienen como ámbito únicamente el objeto en el que están definidas y no las heredan los hijos del objeto.

peligro

Las devoluciones de llamadas del constructor se limitan únicamente al objeto en el que están definidas. No las heredan los elementos secundarios del objeto.

Si está creando una cadena personalizada o un ejecutable, debe recordar propagar las devoluciones de llamadas en tiempo de solicitud a todos los objetos secundarios.

Async en Python<=3.10

Cualquier RunnableLambda, a RunnableGenerator, o Toolque invoque otros ejecutables y se ejecute de forma asíncrona en Python<=3.10, tendrá que propagar devoluciones de llamadas a objetos secundarios de forma manual. Esto se debe a que LangChain no puede propagar devoluciones de llamadas automáticamente a objetos secundarios en este caso.

Esta es una razón común por la que es posible que no puedas ver los eventos emitidos desde herramientas o ejecutables personalizados.

Para obtener información específica sobre cómo utilizar devoluciones de llamadas, consulte las guías prácticas pertinentes aquí .

Técnicas

Transmisión

Las llamadas LLM individuales suelen durar mucho más que las solicitudes de recursos tradicionales. Esto se agrava cuando se crean cadenas o agentes más complejos que requieren varios pasos de razonamiento.

Afortunadamente, los LLM generan resultados de forma iterativa, lo que significa que es posible mostrar resultados intermedios razonables antes de que la respuesta final esté lista. Por lo tanto, consumir los resultados tan pronto como estén disponibles se ha convertido en una parte vital de la experiencia de usuario en torno a la creación de aplicaciones con LLM para ayudar a aliviar los problemas de latencia, y LangChain apunta a tener un soporte de primera clase para la transmisión.

A continuación, analizaremos algunos conceptos y consideraciones sobre la transmisión en LangChain.

.stream() y .astream()

La mayoría de los módulos de LangChain incluyen el .stream() método (y el .astream() método equivalente para entornos asíncronos) como una interfaz de transmisión ergonómica. .stream() devuelve un iterador, que puede consumirse con un forbuque simple. Aquí hay un ejemplo con un modelo de chat:

```
from langchain_anthropic import ChatAnthropic
```

```
model = ChatAnthropic(model="claude-3-sonnet-20240229")
```

```
for chunk in model.stream("what color is the sky?"):
    print(chunk.content, end="|", flush=True)
```

Para los modelos (u otros componentes) que no admiten la transmisión de forma nativa, este iterador solo generaría un único fragmento, pero aún podría usar el mismo patrón general al llamarlos. El uso `.stream()` también llamará automáticamente al modelo en modo de transmisión sin la necesidad de proporcionar una configuración adicional.

El tipo de cada fragmento generado depende del tipo de componente; por ejemplo, los modelos de chat rinden `AIMessageChunks`. Debido a que este método es parte del lenguaje de expresión `LangChain`, puede manejar las diferencias de formato de diferentes salidas utilizando un analizador de salida para transformar cada fragmento generado.

Puede consultar esta guía para obtener más detalles sobre cómo utilizar `.stream()`.

`.astream_events()`

Si bien el `.stream()` método es intuitivo, solo puede devolver el valor final generado de su cadena. Esto está bien para llamadas LLM individuales, pero a medida que crea cadenas más complejas de varias llamadas LLM juntas, es posible que desee utilizar los valores intermedios de la cadena junto con el resultado final; por ejemplo, devolver fuentes junto con la generación final al crear una aplicación de chat sobre documentos.

Hay formas de hacer esto usando callbacks o construyendo tu cadena de tal manera que pase valores intermedios al final con algo como `.assign()` llamadas encadenadas, pero `LangChain` también incluye un `.astream_events()` método que combina la flexibilidad de callbacks con la ergonomía de `.stream()`. Cuando se llama, devuelve un iterador que produce varios tipos de eventos que puedes filtrar y procesar según las necesidades de tu proyecto.

A continuación se muestra un pequeño ejemplo que imprime solo eventos que contienen la salida del modelo de chat transmitido:

```
from langchain core.output_parsers import StrOutputParser
from langchain core.prompts import ChatPromptTemplate
from langchain anthropic import ChatAnthropic

model = ChatAnthropic(model="claude-3-sonnet-20240229")

prompt = ChatPromptTemplate.from_template("tell me a joke about {topic}")
parser = StrOutputParser()
chain = prompt | model | parser
```

```

async for event in chain.astream_events({"topic": "parrot"},
version="v2"):
    kind = event["event"]
    if kind == "on_chat_model_stream":
        print(event, end="|", flush=True)

```

Puedes pensarlo aproximadamente como un iterador sobre eventos de devolución de llamada (aunque el formato difiere) y puedes usarlo en casi todos los componentes de LangChain.

Consulte esta guía para obtener información más detallada sobre cómo utilizar `.astream_events()`, incluida una tabla que enumera los eventos disponibles.

Callbacks

La forma más básica de transmitir salidas desde los LLM en LangChain es a través del sistema de devoluciones de llamadas. Puede pasar un controlador de devoluciones de llamadas que controle el `on_llm_new_token` evento a los componentes de LangChain. Cuando se invoca ese componente, cualquier modelo LLM o de chat incluido en el componente llama a la devolución de llamada con el token generado. Dentro de la devolución de llamada, puede canalizar los tokens a otro destino, por ejemplo, una respuesta HTTP. También puede controlar el evento para realizar cualquier limpieza necesaria. `on_llm_end`

Puede consultar esta sección de instrucciones para obtener información más específica sobre el uso de devoluciones de llamadas.

Las devoluciones de llamadas fueron la primera técnica de transmisión que se introdujo en LangChain. Si bien son potentes y generalizables, pueden resultar difíciles de manejar para los desarrolladores. Por ejemplo:

- Debe inicializar y administrar explícitamente algún agregador u otro flujo para recopilar resultados.
- El orden de ejecución no está garantizado explícitamente y, en teoría, podría ejecutarse una devolución de llamada después de que `.invoke()` finalice el método.
- Los proveedores a menudo le pedirían que pase un parámetro adicional para transmitir las salidas en lugar de devolverlas todas a la vez.
- A menudo ignorarías el resultado de la llamada al modelo real en favor de los resultados de la devolución de llamada.

Tokens

La unidad que la mayoría de los proveedores de modelos utilizan para medir la entrada y la salida es una unidad llamada token . Los tokens son las unidades básicas que los modelos de lenguaje leen y generan al procesar o producir texto. La definición exacta de un token puede variar según la forma específica en que se entrenó el modelo; por ejemplo, en inglés, un token puede ser una sola palabra como "apple" o una parte de una palabra como "app".

Cuando envías un mensaje a un modelo, las palabras y los caracteres del mensaje se codifican en tokens mediante un tokenizador . Luego, el modelo transmite los tokens de salida generados, que el tokenizador decodifica en texto legible para humanos.

La razón por la que los modelos de lenguaje utilizan tokens en lugar de algo más intuitivo e inmediato como "caracteres" tiene que ver con la forma en que procesan y entienden el texto. En un nivel alto, los modelos de lenguaje predicen iterativamente su siguiente salida generada en función de la entrada inicial y sus generaciones anteriores. Al entrenar el modelo con tokens, los modelos de lenguaje pueden manejar unidades lingüísticas (como palabras o subpalabras) que tienen significado, en lugar de caracteres individuales, lo que facilita que el modelo aprenda y comprenda la estructura del lenguaje, incluida la gramática y el contexto. Además, el uso de tokens también puede mejorar la eficiencia, ya que el modelo procesa menos unidades de texto en comparación con el procesamiento a nivel de caracteres.

Function/tool calling

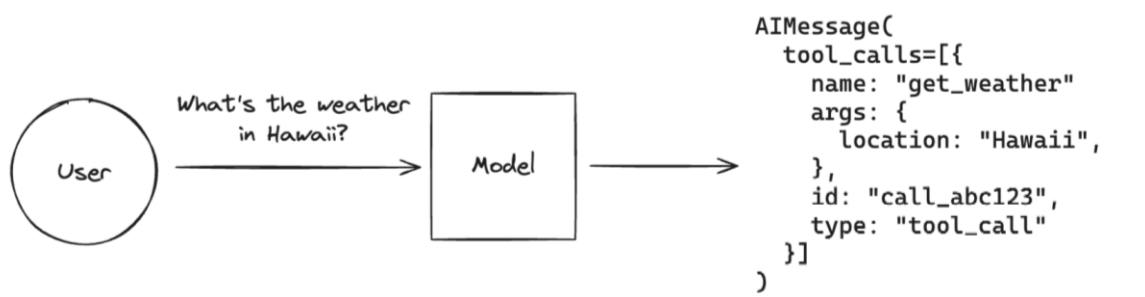
Información

Usamos el término "invocación de herramientas" indistintamente con "invocación de funciones". Si bien a veces la invocación de funciones se refiere a invocaciones de una sola función, tratamos todos los modelos como si pudieran devolver múltiples invocaciones de herramientas o funciones en cada mensaje.

La llamada de herramientas permite que un modelo de chat responda a un mensaje determinado generando una salida que coincide con un esquema definido por el usuario.

Si bien el nombre implica que el modelo está realizando alguna acción, en realidad no es así. El modelo solo genera los argumentos para una herramienta, y la ejecución de la herramienta (o no) depende del usuario. Un ejemplo común en el que no querrías llamar a una función con los argumentos generados es si quieres extraer una salida estructurada que coincida con algún esquema de un texto no estructurado. Le darías al modelo una herramienta de "extracción" que tome los

parámetros que coincidan con el esquema deseado y luego trates la salida generada como tu resultado final.



La llamada de herramientas no es universal, pero cuenta con el respaldo de muchos proveedores de LLM populares, incluidos Anthropic , Cohere , Google , Mistral , OpenAI e incluso para modelos que se ejecutan localmente a través de Ollama .

LangChain proporciona una interfaz estandarizada para llamar herramientas que es consistente en diferentes modelos.

La interfaz estándar consta de:

- **ChatModel.bind_tools()**: un método para especificar qué herramientas están disponibles para que un modelo las llame. Este método acepta herramientas LangChain y objetos Pydantic .
- **AIMessage.tool_calls**: un atributo devuelto AIMessage por el modelo para acceder a las llamadas de herramienta solicitadas por el modelo.

Uso de Tools

Una vez que el modelo llama a las herramientas, puede utilizarlas invocándolas y luego pasando los argumentos de vuelta al modelo. LangChain proporciona la Toolabstracción para ayudarlo a manejar esto.

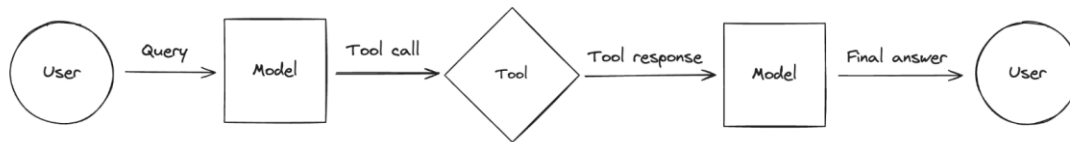
El flujo general es el siguiente:

Generar llamadas a herramientas con un modelo de chat en respuesta a una consulta.

Invoke las herramientas adecuadas utilizando la llamada de herramienta generada como argumentos.

Formatee el resultado de las invocaciones de la herramienta como ToolMessages.

Pase la lista completa de mensajes al modelo para que pueda generar una respuesta final (o llamar más herramientas).



Así es como los agentes que llaman a herramientas realizan tareas y responden consultas.

Salida estructurada

Los LLM pueden generar texto arbitrario. Esto permite que el modelo responda adecuadamente a una amplia gama de entradas, pero para algunos casos de uso, puede ser útil restringir la salida del LLM a un formato o estructura específicos. Esto se conoce como salida estructurada .

Por ejemplo, si la salida se va a almacenar en una base de datos relacional, es mucho más fácil si el modelo genera una salida que se adhiera a un esquema o formato definido. Extraer información específica de texto no estructurado es otro caso en el que esto resulta particularmente útil. Lo más habitual es que el formato de salida sea JSON, aunque otros formatos como YAML también pueden resultar útiles. A continuación, analizaremos algunas formas de obtener una salida estructurada de los modelos en LangChain.

.with_structured_output()

Para mayor comodidad, algunos modelos de chat de LangChain admiten un `.with_structured_output()` método. Este método solo requiere un esquema como entrada y devuelve un diccionario o un objeto Pydantic. Generalmente, este método solo está presente en modelos que admiten uno de los métodos más avanzados que se describen a continuación y utilizarán uno de ellos en segundo plano. Se encarga de importar un analizador de salida adecuado y formatear el esquema en el formato correcto para el modelo.

He aquí un ejemplo:

```
from typing import Optional
```

```
from langchain_core.pydantic_v1 import BaseModel, Field
```

```

class Joke(BaseModel):
    """Joke to tell user."""

    setup: str = Field(description="The setup of the joke")
    punchline: str = Field(description="The punchline to the joke")
    rating: Optional[int] = Field(description="How funny the joke is, from 1 to 10")

structured_llm = llm.with_structured_output(Joke)

structured_llm.invoke("Tell me a joke about cats")

Joke(setup='Why was the cat sitting on the computer?', punchline='To keep an eye on the mouse!', rating=None)

```

Recomendamos este método como punto de partida cuando se trabaja con salida estructurada:

- Utiliza otras características específicas del modelo de forma oculta, sin necesidad de importar un analizador de salida.
- Para los modelos que utilizan llamadas de herramientas, no se necesita ninguna indicación especial.
- Si se admiten varias técnicas subyacentes, puede proporcionar un method parámetro para alternar cuál se utiliza .

Es posible que desee o necesite utilizar otras técnicas si:

- El modelo de chat que estás utilizando no admite la llamada de herramientas.
- Está trabajando con esquemas muy complejos y el modelo está teniendo problemas para generar resultados que se ajusten a los requisitos.

Para obtener más información, consulte esta guía práctica .

También puede consultar esta tabla para obtener una lista de modelos compatibles with_structured_output().

Raw prompting

La forma más intuitiva de conseguir que un modelo estructure la salida es pedirlo amablemente. Además de la consulta, puede dar instrucciones que describan el tipo de salida que desea y luego analizar la salida utilizando un analizador de salida para

convertir el mensaje del modelo sin procesar o la salida de cadena en algo más fácil de manipular.

El mayor beneficio de la indicación en bruto es su flexibilidad:

- La indicación en bruto no requiere ninguna característica especial del modelo, solo la capacidad de razonamiento suficiente para comprender el esquema pasado.
- Puedes solicitar cualquier formato que desees, no solo JSON. Esto puede resultar útil si el modelo que estás usando está más entrenado en un tipo de datos determinado, como XML o YAML.

Sin embargo, también existen algunas desventajas:

- Los LLM no son deterministas, y lograr que un LLM genere datos de manera consistente en el formato exactamente correcto para un análisis sin problemas puede ser sorprendentemente difícil y específico del modelo.
- Los modelos individuales tienen peculiaridades según los datos con los que se entrenaron, y optimizar las indicaciones puede ser bastante difícil. Algunos pueden interpretar mejor el esquema JSON , otros pueden hacerlo mejor con las definiciones TypeScript y otros pueden preferir XML.

Si bien las características ofrecidas por los proveedores de modelos pueden aumentar la confiabilidad, las técnicas de indicación siguen siendo importantes para ajustar los resultados sin importar el método que elija.

JSON mode

Algunos modelos, como Mistral , OpenAI , Together AI y Ollama , admiten una función llamada modo JSON , que generalmente se habilita a través de la configuración.

Cuando está habilitado, el modo JSON restringirá la salida del modelo para que siempre sea algún tipo de JSON válido. A menudo, requieren algún tipo de solicitud personalizada, pero suele ser mucho menos engorroso que una solicitud completamente sin formato y más similar a . "you must always return JSON"La salida también suele ser más fácil de analizar .

También suele ser más sencillo de usar directamente y está más comúnmente disponible que la llamada a herramientas, y puede dar más flexibilidad en cuanto a solicitar y dar forma a los resultados que la llamada a herramientas.

He aquí un ejemplo:

```

from langchain core.prompts import ChatPromptTemplate
from langchain openai import ChatOpenAI
from langchain.output_parsers.json import SimpleJsonOutputParser

model = ChatOpenAI(
    model="gpt-4o",
    model_kwargs={ "response format": { "type": "json object" } },
)

prompt = ChatPromptTemplate.from_template(
    "Answer the user's question to the best of your ability."
    'You must always output a JSON object with an "answer" key and'
    a "followup question" key.'
    "{question}"
)

chain = prompt | model | SimpleJsonOutputParser()

chain.invoke({ "question": "What is the powerhouse of the cell?"
}))

{'answer': 'The powerhouse of the cell is the mitochondrion. It
is responsible for producing energy in the form of ATP through
cellular respiration.',
'followup question': 'Would you like to know more about how
mitochondria produce energy?'}

```

Para obtener una lista completa de proveedores de modelos que admiten el modo JSON, consulte esta tabla .

Tool calling

Para los modelos que lo admiten, la llamada a herramientas puede resultar muy conveniente para obtener resultados estructurados, ya que elimina las dudas sobre cuál es la mejor manera de generar esquemas en favor de una función de modelo integrada.

Funciona vinculando primero el esquema deseado, ya sea directamente o a través de una herramienta LangChain, a un modelo de chat mediante el `.bind_tools()` método. Luego, el modelo generará un `AIMessage` campo `tool_calls` que contenga args los datos que coincidan con la forma deseada.

Existen varios formatos aceptables que se pueden utilizar para vincular herramientas a un modelo en LangChain. A continuación, se muestra un ejemplo:

```

from langchain_core.pydantic_v1 import BaseModel, Field
from langchain_openai import ChatOpenAI

class ResponseFormatter(BaseModel):
    """Always use this tool to structure your response to the
    user."""

    answer: str = Field(description="The answer to the user's
    question")
    followup_question: str = Field(description="A followup
    question the user could ask")

model = ChatOpenAI(
    model="gpt-4o",
    temperature=0,
)

model_with_tools = model.bind_tools([ResponseFormatter])

ai_msg = model_with_tools.invoke("What is the powerhouse of the
cell?")

ai_msg.tool_calls[0]["args"]

{'answer': 'The powerhouse of the cell is the mitochondrion. It
generates most of the cell's supply of adenosine triphosphate
(ATP), which is used as a source of chemical energy.',
'followup_question': 'How do mitochondria generate ATP?'}

```

La llamada de herramientas es una forma generalmente consistente de lograr que un modelo genere una salida estructurada y es la técnica predeterminada utilizada para el `.with_structured_output()` método cuando un modelo lo admite.

Las siguientes guías prácticas son buenos recursos prácticos para utilizar llamadas a funciones o herramientas para obtener resultados estructurados:

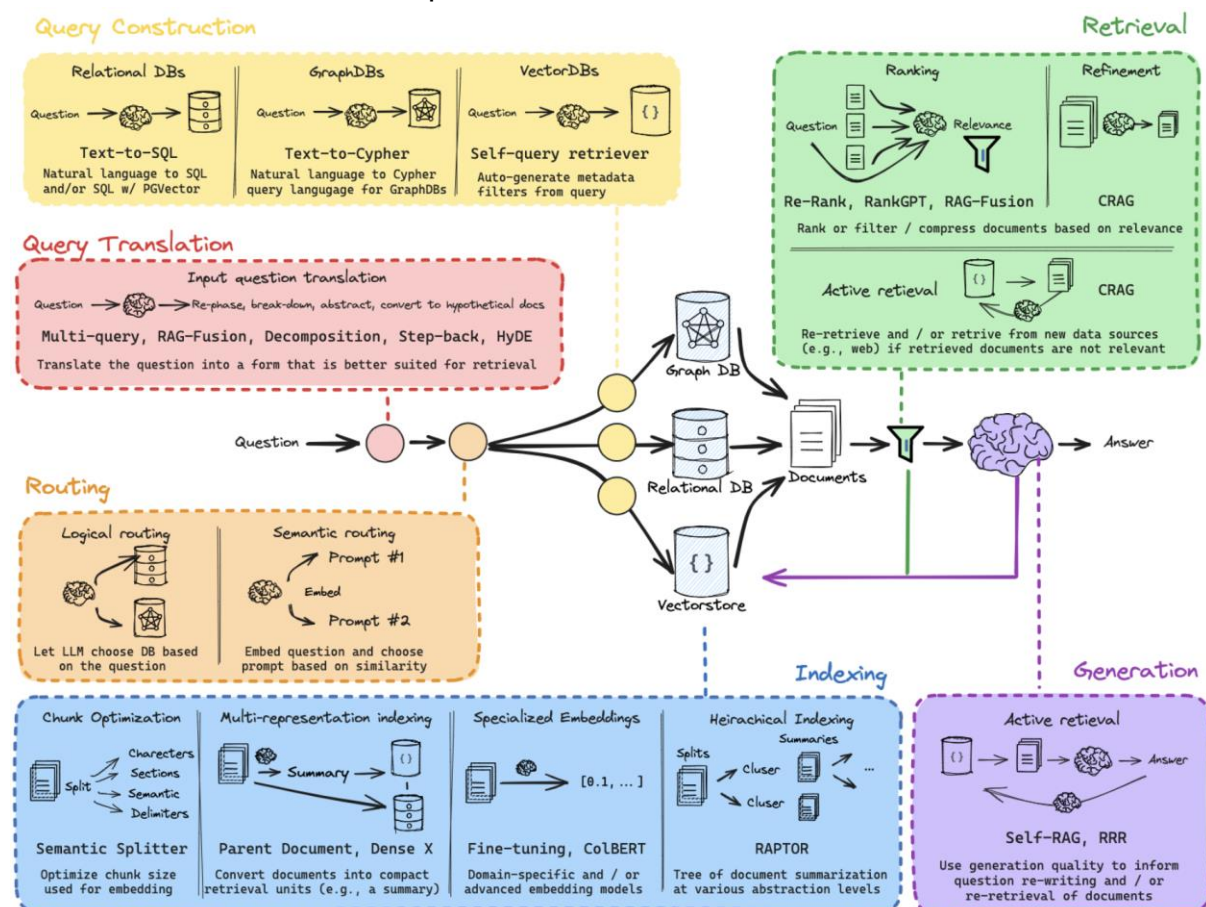
- Cómo devolver datos estructurados desde un LLM
- Cómo utilizar un modelo para llamar a herramientas

Para obtener una lista completa de proveedores de modelos que admiten la llamada de herramientas, consulte esta tabla.

Retrieval

Los LLM se entrenan con un conjunto de datos grande pero fijo, lo que limita su capacidad de razonar sobre información privada o reciente. Ajustar un LLM con hechos específicos es una forma de mitigar esto, pero a menudo no es adecuado para la recuperación de hechos y puede ser costoso. La recuperación es el proceso de proporcionar información relevante a un LLM para mejorar su respuesta para una entrada determinada. La generación aumentada por recuperación (RAG) es el proceso de fundamentar la generación (salida) del LLM utilizando la información recuperada.

La calidad de los documentos recuperados depende de la relevancia y la calidad de los mismos. Afortunadamente, se puede emplear un conjunto emergente de técnicas para diseñar y mejorar los sistemas RAG. Nos hemos centrado en la taxonomía y el resumen de muchas de estas técnicas (consulte la figura siguiente) y compartiremos algunas pautas estratégicas de alto nivel en las siguientes secciones. Puede y debe experimentar con el uso de diferentes piezas juntas. También puede encontrar útil esta guía de LangSmith para mostrar cómo evaluar diferentes iteraciones de su aplicación.



Query Translation

En primer lugar, considere las entradas de los usuarios a su sistema RAG. Idealmente, un sistema RAG puede manejar una amplia gama de entradas, desde preguntas mal redactadas hasta consultas complejas de varias partes. El uso de un LLM para revisar y modificar opcionalmente la entrada es la idea central detrás de la traducción de consultas. Esto sirve como un búfer general, optimizando las entradas de los usuarios sin procesar para su sistema de recuperación. Por ejemplo, esto puede ser tan simple como extraer palabras clave o tan complejo como generar múltiples subpreguntas para una consulta compleja.

Nombre	Cuándo utilizar	Descripción
Multi-query	Cuando necesitas cubrir múltiples perspectivas de una pregunta.	Reescriba la pregunta del usuario desde múltiples perspectivas, recupere documentos para cada pregunta reescrita y devuelva los documentos únicos para todas las consultas.
Decomposition	Cuando una pregunta se puede dividir en subproblemas más pequeños.	Descomponer una pregunta en un conjunto de subproblemas/preguntas, que pueden resolverse secuencialmente (utilizar la respuesta de la primera + la recuperación para responder la segunda) o en paralelo (consolidar cada respuesta en la respuesta final).
Step-back	Cuando se requiere una comprensión conceptual de nivel superior.	En primer lugar, solicite al estudiante de maestría que formule una pregunta genérica de tipo retrospectivo sobre conceptos o principios de nivel superior y recupere datos relevantes sobre ellos. Utilice esta base para ayudar a responder la pregunta del usuario.
HyDE	Si tiene dificultades para recuperar documentos relevantes utilizando las entradas del usuario sin procesar.	Utilice un LLM para convertir preguntas en documentos hipotéticos que respondan a la pregunta. Utilice los documentos hipotéticos integrados para recuperar documentos reales con la premisa de que la búsqueda de similitudes entre documentos puede producir coincidencias más relevantes.

Routing Query

En segundo lugar, considere las fuentes de datos disponibles para su sistema RAG. Desea realizar consultas en más de una base de datos o en fuentes de datos

estructuradas y no estructuradas. El uso de un LLM para revisar la entrada y dirigirla a la fuente de datos adecuada es un enfoque simple y eficaz para realizar consultas en todas las fuentes.

Nombre	Cuándo utilizar	Descripción
Logical routing	Cuando puedes solicitarle a un LLM que establezca reglas para decidir dónde enrutar la entrada.	El enrutamiento lógico puede utilizar un LLM para razonar sobre la consulta y elegir qué almacén de datos es el más apropiado.
Semantic routing	Cuando la similitud semántica es una forma efectiva de determinar dónde enrutar la entrada.	El enrutamiento semántico incorpora tanto una consulta como, por lo general, un conjunto de indicaciones. Luego, elige la indicación adecuada en función de la similitud.

Construction

En tercer lugar, considere si alguna de sus fuentes de datos requiere formatos de consulta específicos. Muchas bases de datos estructuradas utilizan SQL. Los almacenes de vectores suelen tener una sintaxis específica para aplicar filtros de palabras clave a los metadatos de los documentos. El uso de un LLM para convertir una consulta de lenguaje natural en una sintaxis de consulta es un enfoque popular y poderoso. En particular, text-to-SQL , text-to-Cypher y el análisis de consultas para filtros de metadatos son formas útiles de interactuar con bases de datos estructuradas, gráficas y vectoriales respectivamente.

Nombre	Cuándo utilizar	Descripción
Text to SQL	Si los usuarios hacen preguntas que requieren información alojada en una base de datos relacional, accesible a través de SQL.	Esto utiliza un LLM para transformar la entrada del usuario en una consulta SQL.
Text-to-Cypher	Si los usuarios hacen preguntas que requieren información alojada en una base de	Esto utiliza un LLM para transformar la entrada del usuario en una consulta Cypher.

	datos gráfica, accesible a través de Cypher.	
Self Query	Si los usuarios hacen preguntas que se responden mejor obteniendo documentos en función de metadatos en lugar de por similitud con el texto.	Esto utiliza un LLM para transformar la entrada del usuario en dos cosas: (1) una cadena para buscar semánticamente, (2) un filtro de metadatos para acompañarla. Esto es útil porque muchas veces las preguntas se refieren a los METADATOS de los documentos (no al contenido en sí).

Indexing

En cuarto lugar, considere el diseño de su índice de documentos. Una idea simple y poderosa es desacoplar los documentos que indexa para su recuperación de los documentos que pasa al LLM para su generación. La indexación utiliza con frecuencia modelos de incrustación con almacenes de vectores, que comprimen la información semántica de los documentos en vectores de tamaño fijo .

Muchos enfoques de RAG se centran en dividir los documentos en fragmentos y recuperar un número en función de la similitud con una pregunta de entrada para el LLM. Pero el tamaño y el número de fragmentos pueden ser difíciles de establecer y afectar los resultados si no proporcionan un contexto completo para que el LLM responda una pregunta. Además, los LLM son cada vez más capaces de procesar millones de tokens.

Dos enfoques pueden abordar esta tensión: (1) el recuperador de múltiples vectores que utiliza un LLM para traducir documentos a cualquier formato (por ejemplo, a menudo en un resumen) que sea adecuado para la indexación, pero devuelve documentos completos al LLM para su generación. (2) el recuperador Parent Document incorpora fragmentos de documentos, pero también devuelve documentos completos. La idea es obtener lo mejor de ambos mundos: utilizar representaciones concisas (resúmenes o fragmentos) para la recuperación, pero utilizar los documentos completos para la generación de respuestas.

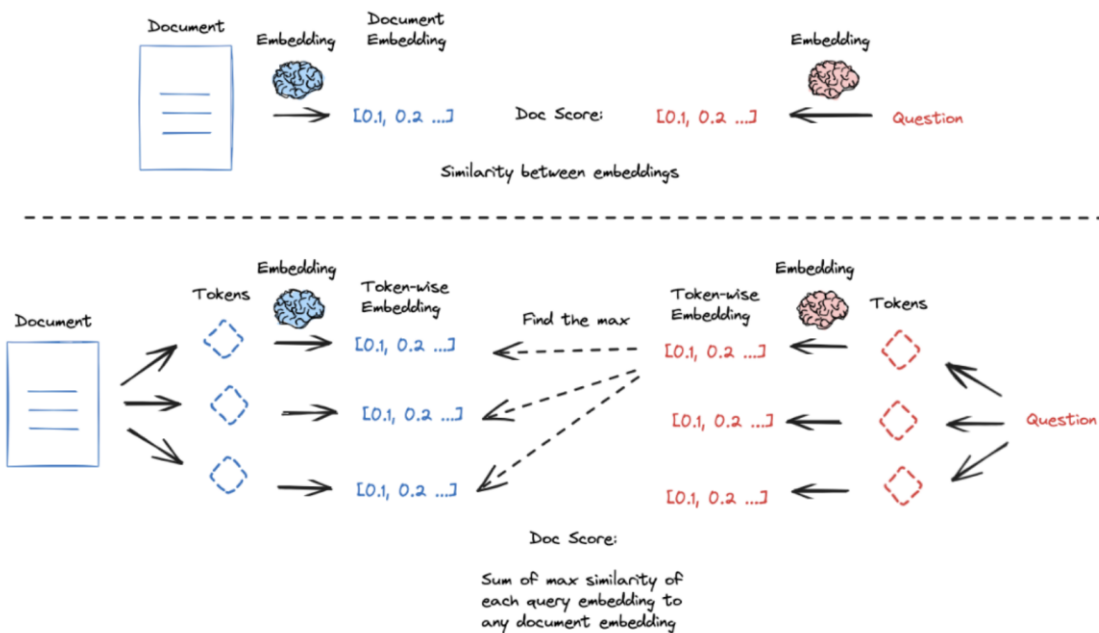
Name	Index Type	Uses an LLM	When to Use	Description
Vector store	Vector store	No	Si recién estás empezando y buscas algo rápido	Este es el método más simple y el más fácil de usar. Consiste en crear

			y fácil.	incrustaciones para cada fragmento de texto.
ParentDocument	Vector store + Document Store	No	Si sus páginas tienen muchos fragmentos pequeños de información distinta que se indexan mejor por sí solos, pero que es mejor recuperarlos todos juntos.	Esto implica indexar varios fragmentos para cada documento. Luego, se buscan los fragmentos que son más similares en cuanto al espacio de inserción, pero se recupera el documento principal completo y se devuelve ese (en lugar de fragmentos individuales).
Multi Vector	Vector store + Document Store	Sometimes during indexing	Si puede extraer información de documentos que considere más relevante para indexar que el texto en sí.	Esto implica crear varios vectores para cada documento. Cada vector se puede crear de muchas maneras, por ejemplo, resúmenes del texto y preguntas hipotéticas.
Time-Weighted Vector store	Vector store	No	Si tiene marcas de tiempo asociadas a sus documentos y desea recuperar las más recientes	Esto recupera documentos basándose en una combinación de similitud semántica (como en la recuperación de vectores normal) y actualidad (mirando las marcas de tiempo de los documentos indexados)

En quinto lugar, considere formas de mejorar la calidad de su propia búsqueda de similitudes. Los modelos de incrustación comprimen el texto en representaciones de longitud fija (vectoriales) que capturan el contenido semántico del documento. Esta compresión es útil para la búsqueda/recuperación, pero impone una gran carga sobre esa única representación vectorial para capturar el matiz/detalle semántico del documento. En algunos casos, el contenido irrelevante o redundante puede diluir la utilidad semántica de la incrustación.

ColBERT es un enfoque interesante para abordar esto con incrustaciones de mayor granularidad: (1) producir una incrustación influenciada contextualmente para cada token en el documento y la consulta, (2) puntuar la similitud entre cada token de

consulta y todos los tokens del documento, (3) tomar el máximo, (4) hacer esto para todos los tokens de consulta y (5) tomar la suma de los puntajes máximos (en el paso 3) para todos los tokens de consulta para obtener un puntaje de similitud de consulta-documento; esta puntuación token por token puede producir resultados sólidos.



Existen algunos trucos adicionales para mejorar la calidad de la recuperación. Las incrustaciones son excelentes para capturar información semántica, pero pueden tener dificultades con las consultas basadas en palabras clave. Muchos almacenes de vectores ofrecen una búsqueda híbrida integrada para combinar la similitud semántica y de palabras clave, lo que combina los beneficios de ambos enfoques. Además, muchos almacenes de vectores tienen una relevancia marginal máxima, que intenta diversificar los resultados de una búsqueda para evitar devolver documentos similares y redundantes.

Nombre	Cuándo utilizar	Descripción
ColBERT	Cuando se necesitan incrustaciones de mayor granularidad.	ColBERT utiliza incrustaciones influenciadas contextualmente para cada token en el documento y la consulta para obtener una puntuación de similitud granular entre la consulta y el documento.
Hybrid search	Al combinar la similitud semántica y basada en palabras clave.	La búsqueda híbrida combina palabras clave y similitud semántica, combinando los beneficios de ambos enfoques.
Maximal Marginal	Cuando es necesario diversificar los	MMR intenta diversificar los resultados de una búsqueda para evitar devolver

Relevance (MMR)	resultados de búsqueda.	documentos similares y redundantes.
-----------------	-------------------------	-------------------------------------

Post-processing

En sexto lugar, considere formas de filtrar o clasificar los documentos recuperados. Esto es muy útil si está combinando documentos recuperados de múltiples fuentes , ya que puede clasificar por debajo de la clasificación los documentos menos relevantes y/o comprimir documentos similares .

Name	Index Type	Uses an LLM	When to Use	Description
Contextual Compression	Any	Sometimes	Si descubre que los documentos recuperados contienen demasiada información irrelevante y distraen el LLM.	Esto agrega un paso de posprocesamiento a otro recuperador y extrae solo la información más relevante de los documentos recuperados. Esto se puede hacer con incrustaciones o un LLM.
Ensemble	Any	No	Si tiene varios métodos de recuperación y desea intentar combinarlos.	Esto recupera documentos de múltiples recuperadores y luego los combina.
Re-ranking	Any	Yes	Si desea clasificar los documentos recuperados según su relevancia, especialmente si desea combinar resultados de múltiples métodos de recuperación.	Dada una consulta y una lista de documentos, Rerank indexa los documentos del más al menos semánticamente relevante para la consulta.

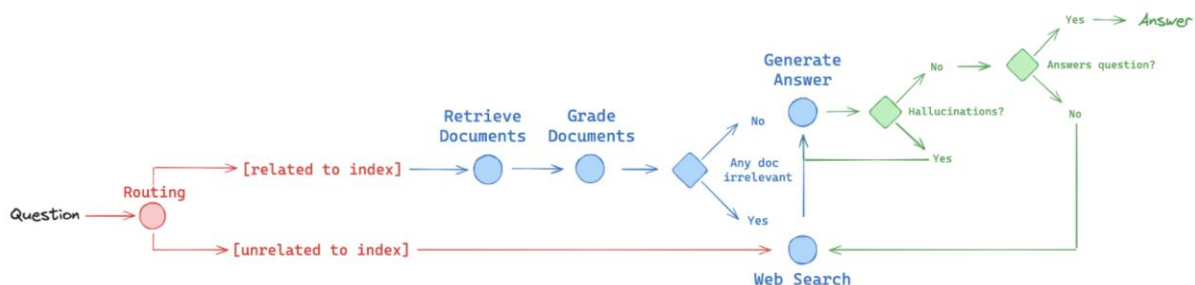
Generation

Por último, considere formas de incorporar la autocorrección en su sistema RAG. Los sistemas RAG pueden sufrir una recuperación de baja calidad (por ejemplo, si

una pregunta de un usuario está fuera del dominio del índice) y/o alucinaciones en la generación. Una secuencia de recuperación-generación ingenua no tiene la capacidad de detectar o autocorregir este tipo de errores. El concepto de "ingeniería de flujo" se ha introducido en el contexto de la generación de código : construir iterativamente una respuesta a una pregunta de código con pruebas unitarias para verificar y autocorregir errores. Varios trabajos han aplicado este RAG, como Self-RAG y Corrective-RAG. En ambos casos, las verificaciones de relevancia del documento, alucinaciones y/o calidad de la respuesta se realizan en el flujo de generación de respuestas RAG.

Descubrimos que los gráficos son una excelente manera de expresar de manera confiable flujos lógicos y hemos implementado ideas de varios de estos artículos utilizando LangGraph , como se muestra en la siguiente figura (rojo: enrutamiento, azul: respaldo, verde: autocorrección):

- **Enrutamiento:** RAG adaptativo (documento). Enrutamiento de preguntas hacia diferentes enfoques de recuperación, como se explicó anteriormente
- **Respaldo:** RAG correctivo (documento). Respaldo a la búsqueda web si los documentos no son relevantes para la consulta
- **Autocorrección:** Autocorrección (papel). Corrija las respuestas con alucinaciones o no responda a la pregunta.



Nombre	Cuándo utilizar	Descripción
Auto-RAG	Cuando sea necesario corregir respuestas con alucinaciones o contenido irrelevante.	Self-RAG realiza comprobaciones de relevancia del documento, alucinaciones y calidad de la respuesta durante el flujo de generación de respuestas RAG, construyendo iterativamente una respuesta y autocorrigiendo errores.
Corrective-RAG	Cuando se necesita un mecanismo de respaldo para documentos de baja relevancia.	Corrective-RAG incluye una alternativa (por ejemplo, una búsqueda web) si los documentos recuperados no son relevantes para la consulta, lo que garantiza una mayor calidad y una recuperación más relevante.

Text splitting

LangChain ofrece muchos tipos diferentes de text splitters. Todos ellos se encuentran en el langchain-text-splitterspaquete.

Columnas de la tabla:

- **Nombre:** Nombre del divisor de texto
- **Clases:** Clases que implementan este divisor de texto
- **Splits On:** Cómo este divisor de texto divide el texto
- **Agrega metadatos:** si este divisor de texto agrega o no metadatos sobre el origen de cada fragmento.
- **Descripción:** Descripción del divisor, incluida recomendación sobre cuándo utilizarlo.

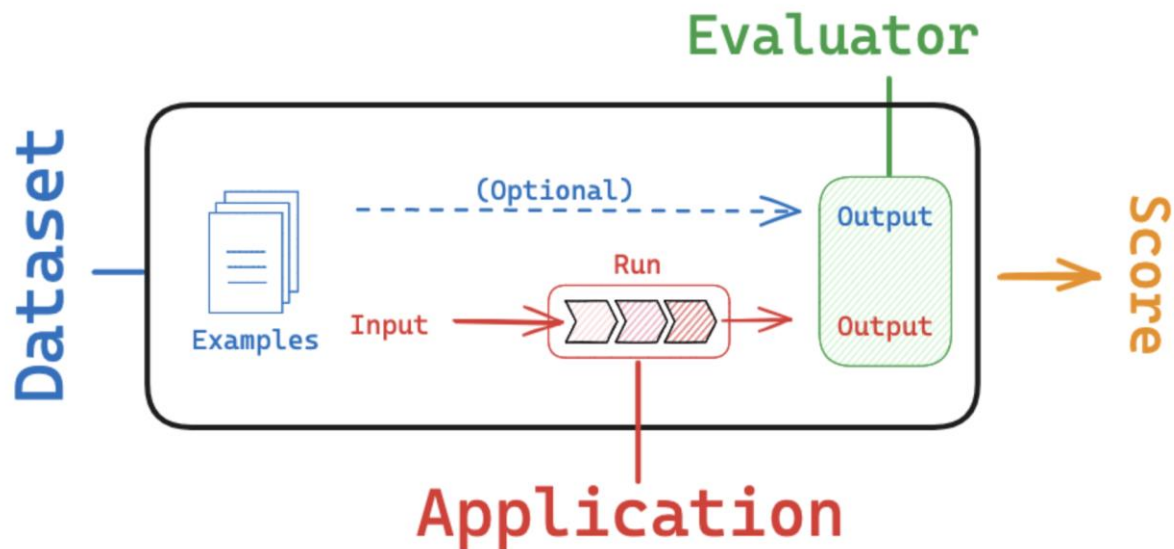
Name	Classes	Splits On	Adds Metadata	Description
Recursive	RecursiveCharacterTextSplitter, RecursiveJsonSplitter	A list of user defined characters		Divide el texto de forma recursiva. Esta división intenta mantener los fragmentos de texto relacionados uno al lado del otro. Esta es la forma recommended way de comenzar a dividir el texto.
HTML	HTMLHeaderTextSplitter, HTMLSectionSplitter	HTML specific characters	✓	Divide el texto en función de caracteres específicos de HTML. Cabe destacar que esto agrega información relevante sobre el origen de ese fragmento (según el HTML).
Markdown	MarkdownHeaderTextSplitter,	Markdown specific characters	✓	Divide el texto en función de caracteres específicos de Markdown. Cabe destacar que esto

				agrega información relevante sobre el origen de ese fragmento (según Markdown)
Code	many languages	Code (Python, JS) specific characters		Divide el texto en función de caracteres específicos de los lenguajes de codificación. Hay 15 idiomas diferentes para elegir.
Token	many classes	Tokens		Divide el texto en tokens. Existen diferentes formas de medir tokens.
Character	CharacterTextSplitter	A user defined character		Divide el texto en función de un carácter definido por el usuario. Uno de los métodos más simples.
Semantic Chunker (Experimental)	SemanticChunker	Sentences		Primero, divide las oraciones. Luego, combina las que están una al lado de la otra si son semánticamente lo suficientemente similares. Tomado de Greg Kamradt
Integration: AI21 Semantic	AI21SemanticTextSplitter		✓	Identifica temas distintos que forman fragmentos de texto coherentes y los divide a lo largo de ellos.

Evaluación

La evaluación es el proceso de evaluar el rendimiento y la eficacia de las aplicaciones basadas en LLM. Implica probar las respuestas del modelo en relación con un conjunto de criterios o puntos de referencia predefinidos para garantizar que

cumple con los estándares de calidad deseados y cumple el propósito previsto. Este proceso es vital para crear aplicaciones confiables.



LangSmith ayuda con este proceso de varias maneras:

- Facilita la creación y conservación de conjuntos de datos a través de sus funciones de seguimiento y anotación.
- Proporciona un marco de evaluación que le ayuda a definir métricas y ejecutar su aplicación en relación con su conjunto de datos.
- Le permite realizar un seguimiento de los resultados a lo largo del tiempo y ejecutar automáticamente sus evaluadores según un cronograma o como parte de CI/Código.

Para obtener más información, consulte esta guía de LangSmith.

Rastreo

Un seguimiento es básicamente una serie de pasos que realiza su aplicación para pasar de la entrada a la salida. Los seguimientos contienen pasos individuales denominados runs. Estos pueden ser llamadas individuales de un modelo, un recuperador, una herramienta o subcadenas. El seguimiento le brinda capacidad de observación dentro de sus cadenas y agentes, y es vital para diagnosticar problemas.

Para profundizar más, consulte esta guía conceptual de LangSmith.

[LangSmith](#)

Evaluación

Generation

Un seguimiento es básicamente una serie de pasos que realiza su aplicación para pasar de la entrada a la salida. Los seguimientos contienen pasos individuales denominados runs. Estos pueden ser llamadas individuales de un modelo, un recuperador, una herramienta o subcadenas. El seguimiento le brinda capacidad de observación

[LangGraph](#)

¿Qué significa ser agente?

Cuando hablamos de sistemas "agenticos", nos referimos a sistemas que utilizan un LLM para decidir el flujo de control de una aplicación. Hay distintos niveles en los que se puede utilizar un LLM para decidir el flujo de control, y este espectro de "agenticos" tiene más sentido para nosotros que definir un límite arbitrario para lo que es o no es un agente.

Ejemplos de uso de un LLM para decidir el control de una aplicación:

- Utilizar un LLM para enrutar entre dos rutas potenciales
- Usar un LLM para decidir a cuál de las muchas herramientas llamar
- Utilizar un LLM para decidir si la respuesta generada es suficiente o se necesita más trabajo

Hay otros conceptos que suelen asociarse con la capacidad de actuar como agente, pero nosotros diríamos que estos son un subproducto de la definición anterior:

- [Llamada a las herramientas](#) : así es como los LLM suelen tomar decisiones

- Toma de acción: a menudo, los resultados de los LLM se utilizan como entrada para una acción.
- **Memoria** : los sistemas confiables necesitan tener conocimiento de las cosas que ocurrieron.
- **Planificación** : los pasos de planificación (ya sea explícita o implícita) son útiles para garantizar que el LLM, al tomar decisiones, las haga con la mayor fidelidad.

¿Por qué LangGraph?

LangGraph tiene varios principios básicos que creemos que lo convierten en el marco más adecuado para crear aplicaciones de agencia:

Controlabilidad

LangGraph es de nivel extremadamente bajo. Esto le brinda un alto grado de control sobre lo que realmente hace el sistema que está construyendo. Creemos que esto es importante porque aún es difícil lograr que los sistemas de agentes funcionen de manera confiable y hemos visto que cuanto más control ejerza sobre ellos, más probabilidades hay de que "funcionen".

El ser humano en el circuito

LangGraph viene con una capa de persistencia incorporada como concepto de primera clase. Esto permite varios patrones de interacción humano-en-el-ciclo diferentes. Creemos que los patrones de "Interacción Humano-Agente" serán la nueva "Interacción Humano-Computadora", y hemos desarrollado LangGraph con persistencia incorporada para permitir esto.

Transmisión primero

LangGraph incluye soporte de primera clase para streaming. Las aplicaciones de Agentic suelen tardar un tiempo en ejecutarse, por lo que es importante dar al usuario una idea de lo que está sucediendo, y el streaming es una excelente manera de hacerlo. LangGraph admite la transmisión de eventos ([como la llamada a una herramienta](#)) y de [tokens que un LLM puede emitir](#) .

Despliegue

Entonces ya has creado tu objeto LangGraph: ¿ahora qué?

Ahora debe implementarlo. Hay muchas formas de implementar objetos LangGraph y la solución adecuada depende de sus necesidades y del caso de uso. Destacaremos dos formas aquí: usar [LangGraph Cloud](#) o implementar su propia solución.

[LangGraph Cloud](#) es una forma de implementar objetos LangGraph basada en opiniones del equipo LangChain. Consulta la [documentación de LangGraph Cloud](#) para conocer todos los detalles sobre lo que implica y ver si es una buena opción para ti.

Si no es la solución adecuada, es posible que desee implementar su propia implementación. En este caso, le recomendamos usar [FastAPI](#) para configurar un servidor. Luego, puede llamar a este gráfico desde el interior del servidor FastAPI como crea conveniente.

Guía conceptual de bajo nivel.

Graphs

En esencia, LangGraph modela los flujos de trabajo de los agentes como gráficos. Usted define el comportamiento de sus agentes mediante tres componentes clave:

1. [State](#): Una estructura de datos compartida que representa la instantánea actual de su aplicación. Puede ser de cualquier tipo Python, pero normalmente es un TypedDict tipo Python o Pydantic BaseModel.
2. [Nodes](#): Funciones de Python que codifican la lógica de sus agentes. Reciben la corriente State como entrada, realizan algún cálculo o efecto secundario y devuelven un archivo State.
3. [Edges](#): Funciones de Python que determinan cuál Node ejecutar a continuación en función de la condición actual State. Pueden ser ramas condicionales o transiciones fijas.

Al componer [Nodes](#) y [Edges](#), puede crear flujos de trabajo complejos y en bucle que evolucionan con el [State](#) tiempo. Sin embargo, el verdadero poder proviene de cómo LangGraph administra eso [State](#). Para enfatizar: [Nodes](#) y [Edges](#) no son más que funciones de Python; pueden contener un LLM o simplemente el buen y viejo código de Python.

En resumen: los nodos hacen el trabajo. Los Edges indican qué hacer a continuación .

El algoritmo de grafos subyacente de LangGraph utiliza [el paso de mensajes](#) para definir un programa general. Cuando un Node completa su operación, envía mensajes a lo largo de uno o más Edges a otros nodos. Estos nodos receptores ejecutan sus funciones, pasan los mensajes resultantes al siguiente conjunto de nodos y el proceso continúa. Inspirado en el sistema [Pregel](#) de Google, el programa avanza en "superpasos" discretos.

Un superpaso puede considerarse una única iteración sobre los nodos del grafo. Los nodos que se ejecutan en paralelo forman parte del mismo superpaso, mientras que los nodos que se ejecutan secuencialmente pertenecen a superpasos separados. Al comienzo de la ejecución del grafo, todos los nodos comienzan en un `inactive` estado. Un nodo se convierte `active` cuando recibe un nuevo mensaje (estado) en cualquiera de sus Edge entrantes (o "canales"). El nodo activo luego ejecuta su función y responde con actualizaciones. Al final de cada superpaso, los nodos sin mensajes entrantes votan `halt` marcando como `inactive`. La ejecución del grafo termina cuando todos los nodos están `inactive` y no hay mensajes en tránsito.

Grafico de estado

La `StateGraph` clase es la clase gráfica principal que se utiliza. Está parametrizada por un `State` objeto definido por el usuario.

Grafico de mensajes

La `MessageGraph` clase es un tipo especial de gráfico. El `State` de un `MessageGraph` es SOLO una lista de mensajes. Esta clase rara vez se usa, excepto para chatbots, ya que la mayoría de las aplicaciones requieren que `State` sea más complejo que una lista de mensajes.

Copilando el grafico

Para crear un gráfico, primero se define el [estado](#), luego se agregan [nodos](#) y [aristas](#) y, por último, se compila. ¿Qué es exactamente compilar un gráfico y por qué es necesario?

La compilación es un paso bastante simple. Proporciona algunas comprobaciones básicas de la estructura de su gráfico (sin nodos huérfanos, etc.). También es donde puede especificar argumentos de tiempo de ejecución como [puntos de control](#) y [puntos de interrupción](#). Compila su gráfico simplemente llamando al `.compile` método:

```
graph = graph_builder.compile(...)
```

DEBE **compilar** su gráfico antes de poder usarlo.

State

Lo primero que se hace cuando se define un gráfico es definir el `State` del gráfico. El `State` consta del [esquema del gráfico](#), así como [reducer de funciones](#) que especifican cómo aplicar actualizaciones al estado. El esquema del `State` será el esquema de entrada para todos los `Nodes` y `Edges` en el gráfico, y puede ser un `TypedDict` o un `Pydantic` modelo. Todos los `Nodes` emitirán actualizaciones al `State` que luego se aplicarán utilizando la `reducer` función especificada.

Schema

La principal forma documentada de especificar el esquema de un gráfico es mediante `TypedDict`. Sin embargo, también admitimos [el uso de un Pydantic BaseModel](#) como estado del gráfico para agregar **valores predeterminados** y validación de datos adicional.

De forma predeterminada, el gráfico tendrá los mismos esquemas de entrada y salida. Si desea cambiar esto, también puede especificar esquemas de entrada y salida explícitos directamente. Esto es útil cuando tiene muchas claves y algunas son explícitamente para entrada y otras para salida. Consulte el [cuaderno aquí](#) para saber cómo usarlo.

De manera predeterminada, todos los nodos del gráfico compartirán el mismo estado. Esto significa que leerán y escribirán en los mismos canales de estado. Es posible que los nodos escriban en canales de estado privados dentro del gráfico para la comunicación interna entre nodos. Consulta [este cuaderno](#) para saber cómo hacerlo.

Reducers

Los reductores son fundamentales para comprender cómo se aplican las actualizaciones de los nodos a la clave `State`. Cada clave de la `State` clave tiene su propia función reductora independiente. Si no se especifica explícitamente ninguna función reductora, se supone que todas las actualizaciones de esa clave deben anularla. Existen algunos tipos diferentes de reductores, comenzando por el tipo de reductor predeterminado

- **Reductor predeterminado:** Estos dos ejemplos muestran cómo utilizar el reductor predeterminado:

Ejemplo A:

En este ejemplo, no se especifican funciones reductoras para ninguna clave. Supongamos que la entrada al gráfico es `{"foo": 1, "bar": ["hi"]}`. Supongamos entonces que el primer `Node` devuelve `{"foo": 2}`. Esto se trata como una actualización del estado. Observe que el `Node` necesita devolver todo el `State` esquema, solo una actualización. Después de aplicar esta actualización, el `State` sería entonces `{"foo": 2, "bar": ["hi"]}`. Si el segundo nodo devuelve `{"bar": ["bye"]}` entonces el `State` sería entonces `{"foo": 2, "bar": ["bye"]}`

Ejemplo B:

En este ejemplo, hemos utilizado el `Annotated` tipo para especificar una función reductora (`operator.add`) para la segunda clave (`bar`). Tenga en cuenta que la primera clave permanece sin cambios. Supongamos que la entrada al gráfico es `{"foo": 1, "bar": ["hi"]}`. Supongamos entonces que la primera `Node` devuelve `{"foo": 2}`. Esto se trata como una actualización del estado. Tenga en cuenta que `Node` necesita devolver todo el `State` esquema, solo una actualización. Después de aplicar esta actualización, entonces `State` sería `{"foo": 2, "bar": ["hi"]}`. Si el segundo nodo devuelve `{"bar": ["bye"]}` entonces `State` entonces sería `{"foo": 2, "bar": ["hi", "bye"]}`. Observe aquí que la `bar` clave se actualiza sumando las dos listas.

- **Reductor de Contexto:** Puede utilizar `Context` canales para definir recursos compartidos (como conexiones de bases de datos) que se administran fuera de los nodos de su gráfico y se excluyen de los puntos de control. El administrador de contexto proporcionado al canal de contexto se ingresa antes del primer paso de la ejecución del gráfico y se sale después del último paso, lo que le permite configurar y limpiar recursos durante la invocación del gráfico. Lea este [tutorial para](#) ver un ejemplo de cómo usar el `Context` canal en su gráfico.

Trabajar con mensajes en el estado del graph

Porque utilizar mensajes

La mayoría de los proveedores de LLM modernos tienen una interfaz de modelo de chat que acepta una lista de mensajes como entrada. LangChain, `ChatModel` en particular, acepta una lista de `Message` objetos como entradas. Estos mensajes vienen en una variedad de formatos, como `HumanMessage` (entrada del usuario) o `AIMessage` (respuesta de LLM). Para leer más sobre qué son los objetos de mensaje, consulte [esta](#) guía conceptual.

Como usar mensajes en su Graph

En muchos casos, resulta útil almacenar el historial de conversaciones anteriores como una lista de mensajes en el estado del gráfico. Para ello, podemos añadir una clave (canal) al estado del gráfico que almacena una lista de `Message` objetos y anotarla con una función reductora (consulte `messages` la clave en el ejemplo siguiente). La función reductora es fundamental para indicar al gráfico cómo actualizar la lista de `Message` objetos en el estado con cada actualización de estado (por ejemplo, cuando un nodo envía una actualización). Si no especifica un reductor, cada actualización de estado sobrescribirá la lista de mensajes con el valor proporcionado más recientemente. Si simplemente desea agregar mensajes a la lista existente, puede utilizar `operator.add` como reductor.

Sin embargo, es posible que también desees actualizar manualmente los mensajes en tu estado gráfico (por ejemplo, con intervención humana). Si usas `operator.add`, las actualizaciones de estado manuales que envíes al gráfico se agregarían a la lista existente de mensajes, en lugar de actualizar los mensajes existentes. Para evitar eso, necesitas un reductor que pueda realizar un seguimiento de las identificaciones de los mensajes y sobrescribir los mensajes existentes, si se actualizan. Para lograr esto, puedes usar la `add_messages` función preconstruida. Para los mensajes nuevos, simplemente se agregará a la lista existente, pero también manejará las actualizaciones de los mensajes existentes correctamente.

Serialization

Además de realizar un seguimiento de los identificadores de los mensajes, la `add_messages` función también intentará deserializar los mensajes en `Message` objetos LangChain cada vez que se reciba una actualización de estado en el `messages` canal. Consulte más información sobre la serialización/deserialización de LangChain [aquí](#). Esto permite enviar entradas de gráficos/actualizaciones de estado en el siguiente formato:

Dado que las actualizaciones de estado siempre se deserializan en LangChain `Messages` cuando se usa `add_messages`, debe utilizar la notación de puntos para acceder a los atributos de los mensajes, como `state["messages"][-1].content`. A continuación, se muestra un ejemplo de un gráfico que utiliza `add_messages` como función reductora.

MessagesState

Dado que es muy común tener una lista de mensajes en el estado, existe un estado predefinido llamado `MessagesState` que facilita el uso de mensajes. `MessagesState` Se define con una sola `messages` clave que es una lista de `AnyMessage` objetos y utiliza el `add_messages` reductor. Normalmente, hay más estados para rastrear que solo mensajes, por lo que vemos que las personas subclasifican este estado y agregan más campos, como:

Nodes

En LangGraph, los nodos son típicamente funciones de Python (sync o `async`) donde el **primer** argumento posicional es el [estado](#) y (opcionalmente), el **segundo** argumento posicional es una "configuración", que contiene [parámetros configurables](#) opcionales (como un `thread_id`).

De manera similar a `NetworkX`, agrega estos nodos a un gráfico usando el método [add_node](#) :

Detrás de escena, las funciones se convierten a [RunnableLambda](#) , que agrega soporte por lotes y asíncrono a su función, junto con seguimiento y depuración nativos.

Si agrega un nodo al gráfico sin especificar un nombre, se le asignará un nombre predeterminado equivalente al nombre de la función.

Start Node

El `START` nodo es un nodo especial que representa el nodo que envía la entrada del usuario al gráfico. El objetivo principal de hacer referencia a este nodo es determinar qué nodos se deben llamar primero.

```
from langgraph.graph import START
```

```
graph.add_edge(START, "node_a")
```

End Node

El `END` nodo es un nodo especial que representa un nodo terminal. Se hace referencia a este nodo cuando se desea indicar qué bordes no tienen acciones después de que se realizan.

```
from langgraph.graph import END
```

```
graph.add_edge("node_a", END)
```

Edges

Los bordes definen cómo se enruta la lógica y cómo el gráfico decide detenerse. Esto es una parte importante de cómo funcionan los agentes y cómo se comunican los diferentes nodos entre sí. Hay algunos tipos clave de bordes:

- Aristas normales: van directamente de un nodo al siguiente.
- Aristas condicionales: llama a una función para determinar a qué nodo(s) ir a continuación.
- Punto de entrada: qué nodo llamar primero cuando llega la entrada del usuario.
- Punto de entrada condicional: llama a una función para determinar qué nodo(s) llamar primero cuando llega la entrada del usuario.

Un nodo puede tener MÚLTIPLES aristas salientes. Si un nodo tiene múltiples aristas salientes, **todos** esos nodos de destino se ejecutarán en paralelo como parte del siguiente superpaso.

Normal Edge

Si **siempre** desea ir del nodo A al nodo B, puede utilizar directamente el método [add_edge](#) .

```
graph.add_edge("node_a", "node_b")
```

Conditional Edge

Si desea enrutar **opcionalmente** a uno o más bordes (o terminar opcionalmente), puede utilizar el método [add_conditional_edges](#) . Este método acepta el nombre de un nodo y una "función de enrutamiento" para llamar después de que se ejecute ese nodo:

```
graph.add_conditional_edges("node_a", routing_function)
```

Similares a los nodos, `routing_function` aceptan la corriente `state` del gráfico y devuelven un valor.

De manera predeterminada, el valor de retorno `routing_function` se utiliza como el nombre del nodo (o una lista de nodos) al que se enviará el estado a continuación. Todos esos nodos se ejecutarán en paralelo como parte del siguiente superpaso.

Opcionalmente, puede proporcionar un diccionario que asigne la `routing_function` salida de al nombre del siguiente nodo.

```
graph.add_conditional_edges("node_a", routing_function, {True: "node_b", False: "node_c"})
```

Entry point

El punto de entrada es el primer nodo que se ejecuta cuando se inicia el gráfico. Puede utilizar el `add_edge` método desde el nodo virtual `START` hasta el primer nodo que se va a ejecutar para especificar dónde ingresar al gráfico.

```
from langgraph.graph import START
```

```
graph.add_edge(START, "node_a")
```

Conditional Entry point

Un punto de entrada condicional le permite comenzar en diferentes nodos según la lógica personalizada. Puede utilizar `add_conditional_edges` el `START` nodo virtual para lograr esto.

```
from langgraph.graph import START
```

```
graph.add_conditional_edges(START, routing_function)
```

Opcionalmente, puede proporcionar un diccionario que asigne la `routing_function` salida de al nombre del siguiente nodo.

```
graph.add_conditional_edges(START, routing_function, {True: "node_b", False: "node_c"})
```

Send

De manera predeterminada, `Nodes` y `Edges` se definen con anticipación y operan en el mismo estado compartido. Sin embargo, puede haber casos en los que no se conozcan los bordes exactos con anticipación y/o es posible que desee `State` que existan diferentes versiones de al mismo tiempo. Un ejemplo común de esto es con `map-reduce` los patrones de diseño. En este patrón de diseño, un primer nodo puede generar una lista de objetos y es posible que desee aplicar algún otro nodo a todos esos objetos. La cantidad de objetos puede ser desconocida con anticipación (lo que significa que es posible que no se conozca la cantidad de bordes) y la entrada `State` al flujo descendente `Node` debe ser diferente (una para cada objeto generado).

Para respaldar este patrón de diseño, LangGraph admite el retorno `Send` de objetos desde bordes condicionales. `Send` toma dos argumentos: primero es el nombre del nodo y segundo es el estado que se pasará a ese nodo.

```
def continue_to_jokes(state: OverallState):  
  
    return [Send("generate_joke", {"subject": s}) for s in state['subjects']]  
  
graph.add_conditional_edges("node_a", continue_to_jokes)
```

Checkpoint

LangGraph tiene una capa de persistencia incorporada, implementada a través de [puntos de control](#) . Cuando se utiliza un punto de control con un gráfico, se puede interactuar con el estado de ese gráfico. Cuando se utiliza un punto de control con un gráfico, se puede interactuar con el estado del gráfico y administrarlo. El punto de control guarda un *punto de control* del estado del gráfico en cada superpaso, lo que permite varias capacidades potentes:

En primer lugar, los puntos de control facilitan [los flujos de trabajo con intervención humana](#) al permitir que los humanos inspeccionen, interrumpan y aprueben pasos. Los puntos de control son necesarios para estos flujos de trabajo, ya que el humano debe poder ver el estado de un gráfico en cualquier momento y el gráfico debe poder reanudar la ejecución después de que el humano haya realizado alguna actualización al estado.

En segundo lugar, permite la ["memoria"](#) entre interacciones. Puedes usar puntos de control para crear hilos y guardar el estado de un hilo después de que se ejecute un gráfico. En el caso de interacciones humanas repetidas (como conversaciones), cualquier mensaje de seguimiento se puede enviar a ese punto de control, que conservará su memoria de los mensajes anteriores.

Consulte [esta guía](#) para saber cómo agregar un puntero de control a su gráfico.

Threads

Los subprocesos permiten la creación de puntos de control para varias ejecuciones diferentes, lo que los hace esenciales para aplicaciones de chat de múltiples inquilinos y otros escenarios en los que es necesario mantener estados separados. Un subproceso es una identificación única asignada a una serie de puntos de control guardados por un puntero de control. Al usar un puntero de control, debe especificar un `thread_id` o `thread_ts` al ejecutar el gráfico.

`thread_id`es simplemente el ID de un hilo. Esto siempre es obligatorio.

`thread_ts`Se puede pasar opcionalmente. Este identificador hace referencia a un punto de control específico dentro de un hilo. Se puede utilizar para iniciar la ejecución de un gráfico desde algún punto a mitad de un hilo.

Debes pasarlos al invocar el gráfico como parte de la parte configurable de la configuración.

```
config = {"configurable": {"thread_id": "a"}}
```

```
graph.invoke(inputs, config=config)
```

Consulte [esta guía](#) para saber cómo utilizar hilos.

Checkpoint State

Al interactuar con el estado del puntero de control, debe especificar un [identificador de hilo](#) . Cada punto de control guardado por el puntero de control tiene dos propiedades:

- **valores** : Este es el valor del estado en este momento.
- **siguiente** : Esta es una tupla de los nodos que se ejecutarán a continuación en el gráfico.

Obtener el estado

Puedes obtener el estado de un puntero de control llamando a `graph.get_state(config)`. La configuración debe contener `thread_id`, y se obtendrá el estado para ese subproceso.

Obtener el historial del estado

También puedes llamar `graph.get_state_history(config)`para obtener una lista del historial del gráfico. La configuración debe contener `thread_id` y se obtendrá el historial de estado para ese hilo.

Actualizar el estado

También puedes interactuar con el estado directamente y actualizarlo. Esto requiere tres componentes diferentes:

- **configuración:** La configuración debe contener `thread_id` la especificación de qué hilo actualizar.
- **Valores:** Estos son los valores que se utilizarán para actualizar el estado. Tenga en cuenta que esta actualización se trata exactamente como se trata cualquier actualización de un nodo. Esto significa que estos valores se pasarán a las funciones [reductoras](#) que forman parte del estado. Por lo tanto, esto NO sobrescribe automáticamente el estado. Veamos un ejemplo.

Supongamos que ha definido el estado de su gráfico como:

Supongamos ahora que el estado actual del gráfico es

```
{"foo": 1, "bar": ["a"]}
```

Si actualiza el estado como se muestra a continuación:

```
graph.update_state(config, {"foo": 2, "bar": ["b"]})
```

Entonces el nuevo estado del gráfico será:

```
{"foo": 2, "bar": ["a", "b"]}
```

La `foo` clave se modifica por completo (porque no hay ningún reductor especificado para esa clave, por lo que la sobrescribe). Sin embargo, hay un reductor especificado para la `bar` clave, por lo que se agrega "b" al estado de `bar`.

- **as_node:** Lo último que se especifica al llamar `update_state` es `as_node`. Esta actualización se aplicará como si viniera del nodo `as_node`. Si `as_node` no se proporciona, se establecerá en el último nodo que actualizó el estado, si no es ambiguo.

La razón por la que esto es importante es que los siguientes pasos a ejecutar en el gráfico dependen del último nodo que haya proporcionado una actualización, por lo que esto se puede usar para controlar qué nodo se ejecuta a continuación.

Common Agentic Patterns

Salida estructurada

Es bastante común querer que los LLM dentro de los nodos devuelvan una salida estructurada al crear agentes. Esto se debe a que esa salida estructurada a menudo

se puede usar para enrutar al siguiente paso (por ejemplo, elegir entre dos bordes diferentes) o actualizar claves específicas del estado.

Dado que los nodos de LangGraph pueden ser funciones de Python arbitrarias, puedes hacer esto como quieras. Si quieres usar LangChain, [esta guía práctica](#) es un punto de partida.

Llamada a las tolos

Es muy común querer que los agentes realicen llamadas a herramientas. La llamada a herramientas se refiere a elegir entre varias herramientas disponibles y especificar cuáles llamar y cuáles deberían ser las entradas. Esto es muy común en los agentes, ya que a menudo se desea dejar que el LLM decida qué herramientas llamar y luego llamar a esas herramientas.

Dado que los nodos de LangGraph pueden ser funciones de Python arbitrarias, puedes hacer esto como quieras. Si quieres usar LangChain, [esta guía práctica](#) es un punto de partida.

Memoria

La memoria es un concepto clave para las aplicaciones de agentes. La memoria es importante porque los usuarios finales a menudo esperan que la aplicación con la que interactúan recuerde interacciones anteriores. El ejemplo más simple de esto son los chatbots: claramente necesitan recordar mensajes anteriores en una conversación.

LangGraph es ideal para brindarle un control total sobre la memoria de su aplicación. Con la opción definida por el usuario, `State` puede especificar el esquema exacto de la memoria que desea conservar. Con [los puntos de control](#), puede almacenar puntos de control de interacciones anteriores y reanudar desde allí las interacciones posteriores.

Consulte [esta guía](#) para saber cómo agregar memoria a su gráf

The humman in the loop

Los sistemas de agentes a menudo requieren algunos patrones de interacción de humanos en el circuito (o "en el circuito"). Esto se debe a que los sistemas de agentes aún no son muy confiables, por lo que es necesario que haya un humano involucrado para cualquier tarea o acción sensible. Todos estos patrones se habilitan fácilmente en LangGraph, en gran parte debido a los [puntos de control](#). La razón por la que es necesario un punto de control es que muchos de estos patrones de interacción implican ejecutar un gráfico hasta cierto punto, esperar algún tipo de

respuesta humana y luego continuar. Cuando desee "continuar", deberá acceder al estado del gráfico antes de ser interrumpido, y los puntos de control son una forma incorporada y muy conveniente de hacerlo.

Hay algunos patrones comunes de interacción humana que estamos viendo emerger.

Aprobacion

Una de las opciones básicas es que el agente espere la aprobación antes de ejecutar determinadas herramientas. Pueden ser todas las herramientas o solo un subconjunto de ellas. Esto se recomienda generalmente para acciones más sensibles (como escribir en una base de datos). Esto se puede hacer fácilmente en LangGraph estableciendo un [punto de interrupción](#) antes de nodos específicos.

Consulte [esta guía](#) para saber cómo hacer esto en LangGraph.

Esperar la entrada

Una opción similar es hacer que el agente espere la intervención humana. Esto se puede hacer de la siguiente manera:

1. Crear un nodo específicamente para la entrada humana
2. Agregar un punto de interrupción antes del nodo
3. Obtener la entrada del usuario
4. Actualizar el estado con esa entrada del usuario, actuando como ese nodo
5. Reanudar ejecución

Consulte [esta guía](#) para saber cómo hacer esto en LangGraph.

Editar acciones del agente

Este es un patrón de interacción más avanzado. En este patrón de interacción, el humano puede editar algunas de las decisiones anteriores del agente. Esto puede hacerse durante el flujo (después de un [punto de interrupción](#) , parte del flujo [de aprobación](#)) o después del hecho (como parte del [viaje en el tiempo](#)).

Consulte [esta guía](#) para saber cómo hacer esto en LangGraph.

Viajes en el tiempo

Este es un patrón de interacción bastante avanzado. En este patrón de interacción, el humano puede volver a consultar la lista de puntos de control anteriores,

encontrar uno que le guste, [editar](#)lo opcionalmente y luego reanudar la ejecución desde allí.

Consulte [esta guía](#) para saber cómo hacer esto en LangGraph.

Review llamada a las Tools

Este es un tipo específico de interacción humana, pero vale la pena mencionarlo porque es muy común. Muchas decisiones de los agentes se toman a través de llamadas a herramientas, por lo que es útil tener una experiencia de usuario clara para revisar las llamadas a herramientas.

Una llamada a una herramienta consta de: - El nombre de la herramienta a llamar - Argumentos para pasar a la herramienta

Tenga en cuenta que estas llamadas a herramientas se pueden utilizar para llamar a funciones, pero también se pueden utilizar para otros fines, como para enrutar al agente en una dirección específica. Deberá revisar la llamada a herramientas para ambos casos de uso.

Al revisar las llamadas a herramientas, hay algunas acciones que es posible que desee realizar.

1. Aprobar la llamada a la herramienta (y dejar que el agente continúe su camino)
2. Cambie manualmente la llamada a la herramienta, ya sea el nombre de la herramienta o los argumentos de la herramienta (y deje que el agente continúe su camino después de eso)
3. Deje comentarios sobre la llamada a la herramienta. Esto se diferencia de (2) en que no está modificando la llamada a la herramienta directamente, sino que está dejando comentarios en lenguaje natural que sugieren que el LLM la llame de manera diferente (o que llame a una herramienta diferente). Puede hacer esto agregando un `ToolMessage` haciendo que los comentarios sean el resultado de la llamada a la herramienta, o agregando un `ToolMessage`(que simula un error) y luego un `HumanMessage`(con los comentarios).

Consulte [esta guía](#) para saber cómo hacer esto en LangGraph.

Map-reducer

Un patrón común en los agentes es generar una lista de objetos, realizar algún trabajo en cada uno de esos objetos y luego combinar los resultados. Esto es muy similar a la operación común [map-reduce](#) . Esto puede ser complicado por varias razones. Primero, puede ser difícil definir un gráfico estructurado con anticipación porque la longitud de la lista de objetos puede ser desconocida. Segundo, para hacer este map-reduce, se necesitan múltiples versiones del estado... pero el gráfico comparte un estado compartido común, entonces, ¿cómo puede ser esto posible?

LangGraph admite esto a través de la API [Send](#) . Esto se puede utilizar para permitir que un borde condicional envíe varios estados diferentes a varios nodos. El estado que envía puede ser diferente del estado del gráfico principal.

Vea una guía práctica para esto [aquí](#)

Multiagente

Un término que probablemente hayas escuchado es arquitecturas "multiagente". ¿Qué significa exactamente?

Dado que es difícil incluso definir un "agente", es casi imposible definir con exactitud una arquitectura "multiagente". Cuando la mayoría de las personas hablan de una arquitectura multiagente, normalmente se refieren a un sistema en el que hay varios sistemas diferentes basados en LLM. Estos sistemas basados en LLM pueden ser tan simples como un mensaje y una llamada LLM, o tan complejos como un [agente ReAct](#) .

La gran pregunta en los sistemas multiagente es cómo se comunican. Esto implica tanto el esquema de cómo se comunican, como la secuencia en la que se comunican. LangGraph es perfecto para orquestar este tipo de sistemas. Permite definir múltiples agentes (cada uno es un nodo), un estado arbitrario (para encapsular el esquema de cómo se comunican) así como los bordes (para controlar la secuencia en la que se comunican).

Planificación

Una de las principales dificultades de los sistemas de agentes es la planificación a largo plazo. Una técnica habitual para superarla es contar con una planificación explícita. Esto generalmente implica llamar a un LLM para que elabore una serie de pasos a ejecutar. A partir de ahí, el sistema intenta ejecutar la serie de tareas (para ello, puede utilizar un subagente). Opcionalmente, puede revisar el plan después de cada paso y actualizarlo si es necesario.

Reflexion

A menudo, los agentes tienen dificultades para generar resultados confiables. Por lo tanto, puede ser útil verificar si el agente completó una tarea correctamente o no. Si es así, puede finalizarla. Si no es así, puede tomar los comentarios sobre por qué no es correcta y pasarlos a otra iteración del agente.

Este paso de "reflexión" suele utilizar un LLM, pero no es obligatorio. Un buen ejemplo de un caso en el que el uso de un LLM puede no ser necesario es en la codificación, cuando se puede intentar compilar el código generado y utilizar los errores como retroalimentación.

Agente ReAct

Una de las arquitecturas de agente más comunes es la que comúnmente se denomina arquitectura de agente ReAct. En esta arquitectura, se llama a un LLM repetidamente en un bucle while. En cada paso, el agente decide qué herramientas llamar y cuáles deberían ser las entradas a esas herramientas. Luego, se ejecutan esas herramientas y las salidas se devuelven al LLM como observaciones. El bucle while finaliza cuando el agente decide que no vale la pena llamar a más herramientas.

Uno de los pocos agentes prediseñados de alto nivel que tenemos en LangGraph: puedes usarlo con [create_react_agent](#)

Este documento lleva el nombre del artículo [de ReAct](#) y está basado en él. Sin embargo, existen varias diferencias entre este artículo y nuestra implementación:

- En primer lugar, utilizamos [la llamada a herramientas](#) para que los LLM llamen a las herramientas, mientras que en el artículo se utilizó la indicación + análisis de la salida sin procesar. Esto se debe a que la llamada a herramientas no existía cuando se escribió el artículo, pero en general es mejor y más confiable.

- En segundo lugar, utilizamos mensajes para activar el LLM, mientras que en el artículo se utilizaba el formato de cadena. Esto se debe a que, en el momento de redactar este artículo, los LLM ni siquiera exponían una interfaz basada en mensajes, mientras que ahora esa es la única interfaz que exponen.
- En tercer lugar, el documento exigía que todas las entradas de las herramientas fueran una sola cadena. Esto se debía en gran medida a que los LLM no eran muy capaces en ese momento y solo podían generar una única entrada. Nuestra implementación permite utilizar herramientas que requieren múltiples entradas.
- En cuarto lugar, el documento solo analiza la posibilidad de llamar a una sola herramienta a la vez, en gran medida debido a las limitaciones en el rendimiento de LLM en ese momento. Nuestra implementación permite llamar a varias herramientas a la vez.
- Por último, el artículo le pidió al LLM que generara explícitamente un paso de "Pensamiento" antes de decidir qué herramientas llamar. Esta es la parte de "Razonamiento" de "ReAct". Nuestra implementación no hace esto de manera predeterminada, en gran parte porque los LLM han mejorado mucho y eso no es tan necesario. Por supuesto, si desea solicitarle que lo haga, sin duda puede hacerlo.

Consulta [esta guía](#) para obtener una explicación completa sobre cómo utilizar el agente ReAct prediseñado.

Salida estructurada

Salida estructurada